



Breaking TSP local search barriers: scalable multi-GPU parallelisation of 4-opt, 5-opt, 6-opt and hybrid variable λ -opt

Wen-Bao Qiao^a, Jean-Charles Créput^d, Nan Wang^a, Kun Meng^a, Abdelkhalek Mansouri^d

^a Computer School, Beijing Information Science & Technology University, 102206, Beijing, China

^b Department of Biomedical Engineering, National Biomedical Imaging Center, College of Future Technology, Peking University, 100871, Beijing, China

^c Beijing Key Laboratory of Cyber Culture and Digital Communication, Beijing Information Science & Technology University, 102206, Beijing, China

^d CIAD UR 7533, Université de Technologie de Belfort-Montbéliard, UTBM, F-90010, Belfort, France

ARTICLE INFO

Keywords:

Parallel 4-opt
Multiple 4-opt moves
Parallel 5-opt
Multiple 5-opt moves
Parallel 6-opt
Multiple 6-opt moves
Variable λ -opt
Multiple variable k -opt moves
GPU

ABSTRACT

The 2-opt, 3-opt, 4-opt, 5-opt, and 6-opt are classical k -opt local search algorithms for the travelling salesman problem (TSP) in combinatorial optimisation. However, these algorithms exhibit two limitations. Firstly, serial execution of these algorithms results in polynomial time complexity, which imposes significant computational burdens for large-scale TSP instances. Secondly, k needs to be specified in advance. To address the first limitation, some researchers have exploited GPU parallelisation to develop GPU-parallel 2-opt and 3-opt algorithms. For the second limitation, some researchers have proposed variable λ -opt algorithms. Notably, the extant literature exhibits a paucity of research concerning the GPU-accelerated computation of 4-opt, 5-opt, 6-opt, and variable λ -opt methodologies. This paper presents parallel implementations of complete 4-opt, 5-opt, 6-opt, and variable λ -opt ($\lambda = 2, 3, 4, 5, 6$) search algorithms, which are designed for execution across multiple GPUs. These parallel implementations adhere to data-parallel and support seamless deployment across an arbitrary number of GPUs. Additionally, this paper presents a serial algorithm that executes multiple variable λ -opt moves on the same TSP tour. The source code is publicly available, and evaluations of these GPU-parallel k -opt algorithms were conducted on standard TSP benchmark instances without incorporating nearest-neighbour information. Experimental tests on 12 TSP instances show that the proposed method exhibits the shortest computational time per iteration during complete 4-opt, 5-opt, 6-opt and variable λ -opt search applied to a global TSP tour while refining a near-optimal solution. For example, the single-GPU-parallel implementation of the complete 4-opt search runs over 1280 times faster than its serial counterpart (135.1 s vs >48 h) for mu1979.tsp. This exceptional speedup leads to the fastest implementation method for optimising a brute-force TSP tour through variable complete λ -opt search applied to the global tour. For example, the single-GPU-parallel increasing variable λ -optimal ($\lambda = 2, 3, 4$) method runs more than 58 times faster than its serial version (0.8312 h vs >48 h). Moreover, given a sufficient number of GPUs, further computational acceleration can be achieved, enabling the determination of 5-optimal and 6-optimal TSP tours within acceptable timeframes.

1. Introduction

The travelling salesman problem (TSP) (Applegate et al., 2011; Johnson & McGeoch, 1997; Laporte, 1992) is one of the combinatorial optimisation problems that have been classified as NP-hard (Garey & Johnson, 1979). A solution to a TSP instance is a permutation of the n cities, where different permutations represent different solutions. Following the permutation order, the salesman visits each city exactly once and returns to the starting city. Symmetric TSP forms an undirected graph where the distance between any two cities is identical. Asymmetric TSP forms a directed graph where paths may not exist or the distances in opposite directions might differ between two nodes.

To optimise an initial TSP tour, one of the most successful methods for generating optimal or near-optimal TSP solutions, the Lin-Kernighan heuristic (LKH) algorithm (Helsgaun, 2000), has an operator called **variable λ -opt** where in each step λ edges of the current tour are replaced by λ new edges in such a way that a shorter tour is achieved.

The **λ -opt** (or **k -opt**) represents a family of local search algorithms, including 2-opt, 3-opt, 4-opt, 5-opt, and 6-opt ... ($n - 1$)-opt, each differing in two aspects:

- Firstly, the computational complexity: In complete k -opt local search, all combinations of k edges are exhaustively checked once to identify one potential k -opt improvement. For example, a complete

* Corresponding author.

E-mail address: wen-bao.qiao@bistu.edu.cn; rapidbao@outlook.com (W.-B. Qiao).

<https://doi.org/10.1016/j.eswa.2025.129110>

Received 27 November 2024; Received in revised form 7 June 2025; Accepted 19 July 2025

Available online 25 July 2025

0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

2-opt search requires $\frac{n(n-1)}{2}$ checks, a complete 3-opt search requires $\frac{n(n-1)(n-2)}{6}$ checks, a complete 4-opt search requires $\frac{n(n-1)(n-2)(n-3)}{24}$ checks, a complete 5-opt search requires $\frac{n(n-1)(n-2)(n-3)(n-4)}{120}$ checks, and a complete 6-opt search requires $\frac{n(n-1)(n-2)(n-3)(n-4)(n-5)}{720}$ checks (Johnson & McGeoch, 1997; Lourenço et al., 2003). As k increases, the number of checks grows factorially, leading to exponential computational complexity.

- Secondly, k -opt re-connection schemes: After removing k edges from the original TSP tour, there exist diverse re-connection schemes to ensure the resulting tour remains valid.

Furthermore, only executing one λ -opt exchange cannot generate an optimal or near-optimal TSP tour. LKH adopts an iterative λ -opt strategy to achieve a state called λ -optimal. A tour is said to be λ -optimal if it is impossible to obtain a shorter tour by replacing any λ of its edges with any other set of λ edges (Helsgaun, 2000). Therefore, given an initial TSP tour, to compute its λ -optimal solution using a pure λ -opt algorithm requires iteratively searching and executing λ -opt on the global TSP tour until no further improving λ -opt move can be found. This procedure includes the following two stages:

- Stage 1. Search and execute valid λ -opt moves on the global TSP tour: Examine all possible λ edge combinations to find λ -opt moves that reduce the tour length while maintaining a valid tour.
- Stage 2. Iteratively repeat Stage 1 until the algorithm cannot find any improving λ -opt move.

The critical challenge lies in the fact that, when the algorithm attempts to optimise a near-optimal TSP, the $O(n^4)$ time complexity of Stage 1 arises from the exponential growth of λ -edge combinations with n . Additionally, for Stage 2, there is no non-trivial upper bound for the number of λ -opt moves needed to achieve a λ -optimal solution (Helsgaun, 2000; Lourenço et al., 2003). This computational bottleneck makes it infeasible to apply higher-order λ -opt algorithms directly to large-scale TSP instances.

When addressing this challenge, LKH (Helsgaun, 2000) considers nearest neighbour information between cities and employs strategies such as Euclidean minimum spanning trees (EMST (Qiao & Créput, 2021)) and tour segmentation methods to reduce the λ -opt search space and computation time.

In this study, λ -opt is performed on the global TSP tour, and multiple Graphics Processing Units (GPUs) are utilised to reduce computation time. Despite advances in parallel 2-opt and 3-opt algorithms, no prior work has addressed GPU-parallel implementations for higher-order λ -opt (4-opt, 5-opt, 6-opt) or execution of hybrid variable λ -opt ($\lambda = 2, 3, 4, 5, 6$) moves across the entire TSP tour. This paper addresses these gaps with the following contributions:

1. GPU-parallel complete 4-opt, 5-opt, 6-opt and variable λ -opt ($\lambda = 2, 3, \dots, 6$) search on the global TSP tour incorporate all valid pure 4-/5-/6-opt reconnection schemes.
2. A linear-time method for executing multiple hybrid variable λ -opt moves ($\lambda = 2, 3, \dots, 6$) on the same TSP tour.
3. Algorithms that are designed with native support for deployment across arbitrary numbers of GPUs.

The paper is organised as follows: Section 2 introduces preliminaries including fixed k -opt and variable λ -opt definitions and concepts; Section 3 reviews related parallel k -opt algorithms; Section 4 presents the proposed 'multiple-GPU-parallel 4-opt, 5-opt, 6-opt and variable λ -opt search with multiple hybrid variable λ -opt moves' algorithm; Section 5 details experimental comparisons and results; and Section 6 provides conclusions.

2. Preliminaries

Given a symmetric Euclidean TSP instance, let V be a set of n cities and $d(u, v) \in \mathbb{R}$ be the distance for each pair of cities $u, v \in V$. Let S de-

note the set of all possible tour solutions (Hamiltonian cycles) of a complete undirected graph (V, E) , where $E = \{\{u, v\} \mid u, v \in V, u \neq v\}$ and $s \in S$ represents a solution. Let $\pi : \{0, \dots, j, j+1, \dots, n-1\} \rightarrow V$ be a bijection. A TSP tour solution $s = (\pi_0, \pi_1, \pi_2, \dots, \pi_j, \pi_{j+1}, \dots, \pi_{n-1})$ represents a permutation of the n cities, where each city π_i is visited in order from $j = 0$ to $j = n-1$. Edge e_j denotes the undirected edge between city π_j and π_{j+1} , ensuring each city is visited exactly once, with $(j+1) \bmod n = 0$ such that when $j = n-1$, the tour returns to the starting city π_0 . The objective is to find a solution $s \in S$ that minimises the cost function $f(s) = \sum_{k=0}^{n-1} d(e_k)$, where $d(e_k)$ is the distance of edge e_k .

2.1. Fixed k -opt

Consider two adjacent TSP solutions $s_i, s_{i+1} \in S$. If s_{i+1} yields a shorter tour than s_i by removing and inserting k edges, this transition is defined as a k -opt move $\mathcal{N}_k^{opq\dots}$. A k -opt move requires examination of the affected cities' tour order. For instance, given two oriented edges (e_o, e_p) and the associated cities $o = \pi_o, o+1 = \pi_{o+1}, p = \pi_p, p+1 = \pi_{p+1}$, the 2-opt algorithm needs to verify whether $d(o, o+1) + d(p, p+1) > d(o, p) + d(o+1, p+1)$ to maintain tour continuity, thereby constituting a valid 2-opt evaluation.

The 3-opt ($\mathcal{N}_3$), 4-opt ($\mathcal{N}_4$), 5-opt ($\mathcal{N}_5$), 6-opt ($\mathcal{N}_6$) and general fixed k -opt ($\mathcal{N}_k$) operations follow analogous principles to 2-opt. An important point to note is that removing k edges from a tour produces k subtours, generating $2^{k-1}(k-1)!$ potential reconnection schemes, though many prove degenerate (Lancia & Dalpasso, 2023). As illustrated in Fig. 1, only 4 of 8 reconnection schemes constitute pure 3-opt moves, with the remainder either being 2-opt variants or invalid. Similarly, Fig. 2 shows merely 25 of 48 schemes qualify as pure 4-opt moves (Lancia & Dalpasso, 2020). In the literature, comprehensive documentation of valid reconnection schemes for 5-opt and 6-opt remains scarce.

Execution of a selected k -opt reconnection scheme may reverse the permutation order of one or more subtours, necessitating subsequent reordering operations.

2.2. Variable λ -opt

Variable λ -opt is described as follows in Helsgaun (2000): At each iteration step the algorithm examines, for ascending values of λ , whether an interchange of λ edges may result in a shorter tour. At each step the algorithm considers a growing set of potential exchanges (starting with $\lambda = 2$). These exchanges are chosen in such a way that a feasible tour may be formed at any stage of the process. If the exploration succeeds in finding a new shorter tour, then the actual tour is replaced with the new tour (Helsgaun, 2000).

3. Related parallel k -opt algorithms

To reduce the computation time of pure k -opt algorithms, researchers have proposed parallel 2-opt and 3-opt algorithms (Karp, 1977; Qiao & Créput, 2017a,b, 2018, 2020; Rios et al., 2018; Rocki & Suda, 2012, 2013; Verhoeven et al., 1995). In this section, the relevant parallel k -opt algorithms are first classified, followed by a brief introduction to the method 'multiple k -opt evaluation multiple k -opt moves' proposed by Qiao and Créput (2020), which is particularly relevant to the research content of this paper.

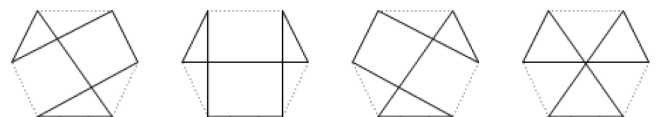


Fig. 1. The four valid pure 3-opt reconnection schemes.

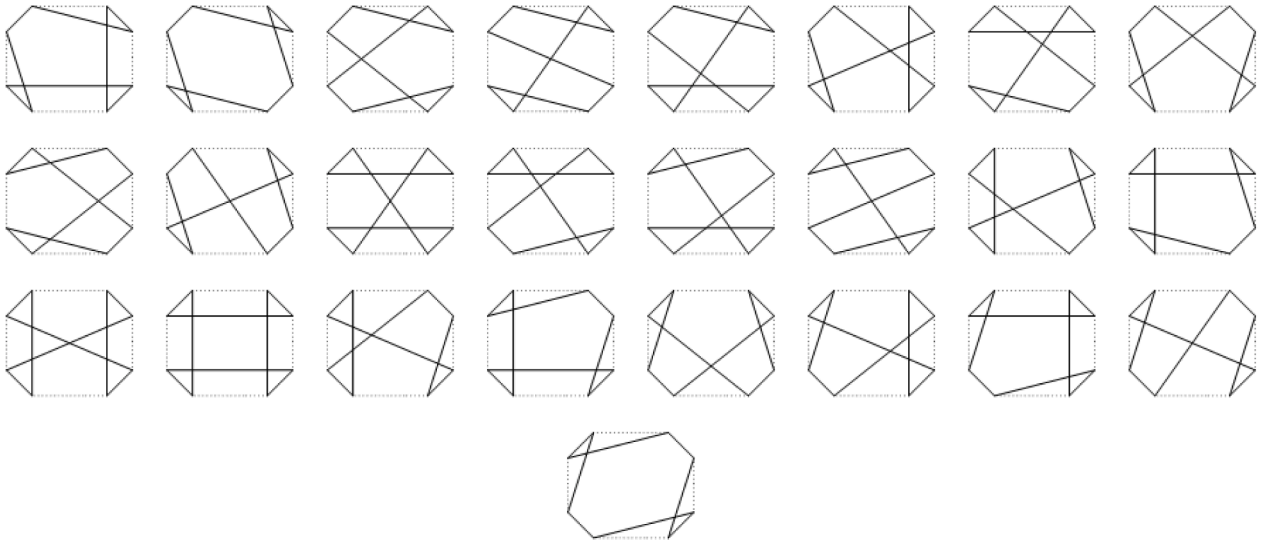


Fig. 2. The 25 valid pure 4-opt reconnection schemes.

3.1. Categories of parallel k -opt algorithms

Existing parallel 2-/3-opt algorithms can be classified into the following three categories:

1. Search and execute multiple 2-/3-opt moves in parallel on disjoint partitions of a single TSP tour (Verhoeven et al., 1995) or in disjoint partitions of the Euclidean space (Karp, 1977; Qiao & Créput, 2017b).
2. Search 2-/3-opt moves in parallel on the global TSP tour, but execute only the best 2-/3-opt move (Rocki & Suda, 2013).
3. Search and execute multiple 2-/3-opt moves in parallel on the global TSP tour or in the global Euclidean space, termed ‘multiple k -opt evaluation multiple k -opt moves’ (Qiao & Créput, 2018, 2020; Rios et al., 2018).

However, none of these existing works studies parallel 4-opt, 5-opt, 6-opt or variable λ -opt to obtain λ -optimal tours when $\lambda \geq 4$. The newly proposed methods in this work, together with the ‘multiple k -opt evaluation multiple k -opt moves’ method (Qiao & Créput, 2018, 2020) from 2018, can be classified as a metaheuristic termed DVND. Rios et al. (2018) proposed the distributed variable neighbourhood descent (DVND) metaheuristic for multi-GPU systems, which includes a neighbourhood search strategy called Multi-Improvement. The core idea of DVND is to utilise massive parallelism on multi-GPU systems to achieve multi-improvement in local search procedures such as 2-opt.

While DVND is a metaheuristic algorithm, for instance, DVND verifies conflicts between candidate moves by examining all remaining moves, resulting in $O(n^2)$ quadratic-time complexity for conflict checking. In this paper, the proposed methods detail the approach to performing GPU parallel λ -opt search on the global TSP tour and to selecting non-conflicting variable λ -opt moves with linear-time complexity.

3.2. Existing multiple k -opt evaluation and moves

The existing ‘multiple k -opt evaluation multiple k -opt moves’ method (Qiao & Créput, 2018, 2020) extends Rocki’s GPU-parallel 2-/3-opt approach (Rocki & Suda, 2012, 2013).

3.2.1. Rocki’s GPU-parallel 2-/3-opt implementation

Rocki’s GPU-parallel algorithm distributes all possible 2- or 3-edge combinations from the global TSP tour across $O(n)$ CUDA threads per iteration (Rocki & Suda, 2013). The computational workload comprises $\frac{n(n-1)}{2}$ 2-opt checks and $\frac{n(n-1)(n-2)}{6}$ 3-opt checks. Fig. 3 illustrates the core

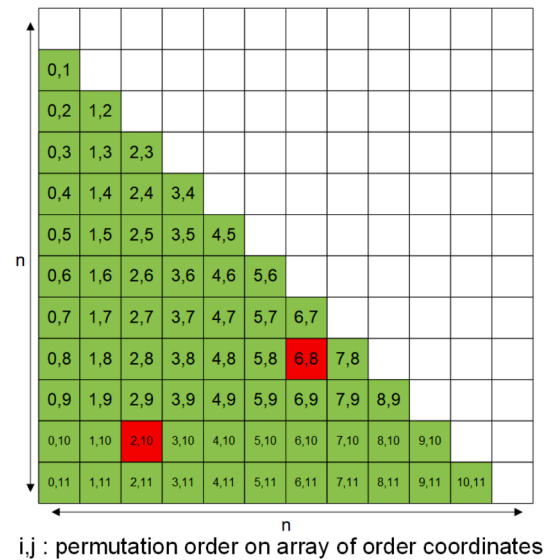


Fig. 3. GPU-parallel 2-opt scheme (Rocki & Suda, 2013) for complete evaluation of $\frac{n(n-1)}{2}$ 2-opt moves on an n -city TSP tour (Qiao & Créput, 2020).

2-opt mechanism, where each cell represents a pair of cities (π_o, π_p) with the cities’ current tour positions. The ordered array data structure ensures consistent orientation for all 2-opt evaluations with respect to the tour direction.

However, Rocki’s implementation exhibits a significant limitation during iterative local optimisation: each iteration evaluates all possible moves but executes only the best 2-/3-opt move. This approach proves inefficient when pursuing λ -optimal tours, as noted in Qiao and Créput (2020).

3.2.2. Multiple k -opt evaluation and moves

The ‘multiple k -opt evaluation multiple k -opt moves’ method in Qiao and Créput (2018, 2020) initially fixes $k = 2$ with a theoretical extension to $k = 3$. This approach builds upon Rocki’s GPU-parallel 2-opt algorithm, introducing a linear-time complexity method for identifying non-interacting 2-opt moves in global TSP tours. By executing multiple non-interacting moves in parallel after each complete 2-opt search iteration (i.e., $\frac{n(n-1)}{2}$ 2-opt checks), the method rapidly achieves 2-optimal TSP solutions.

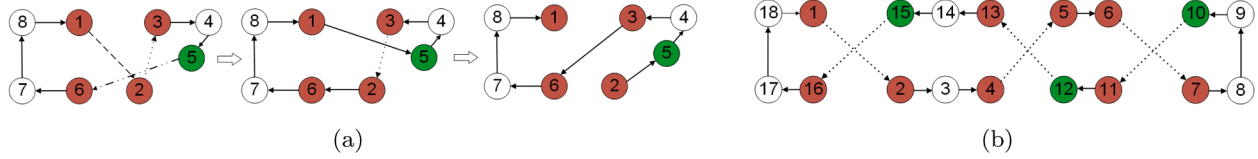


Fig. 4. Multiple 2-opt move scenarios: (a) interacting moves ($\mathcal{N}_2^{1,5} \rightarrow (e_1, e_5)$ and $\mathcal{N}_2^{2,5} \rightarrow (e_2, e_5)$); (b) non-interacting moves. $\textcircled{1}$ denotes the city tour order (Qiao & Créput, 2020).

Its theoretical foundation comprises two key aspects:

- Rocki's GPU-parallel 2-opt inherently identifies a large number of valid independent moves during global tour evaluation.
- As illustrated in Fig. 4, the concurrent execution of interacting 2-opt moves may disrupt tour integrity (Fig. 4(a)), necessitating the proposed algorithm for selecting compatible non-interacting 2-opt moves (Fig. 4(b)).

4. Parallel 4-opt, 5-opt, and 6-opt with multiple variable λ -opt moves

Building upon existing k -opt methodologies, the pursuit of scalable multiple-GPU-parallel variable λ -opt ($2 \leq \lambda \leq 6$) implementation on global TSP tours hinges on resolving four principal obstacles:

- 1 Identification of all valid pure 5-opt and 6-opt reconnection schemes.
- 2 Development of intrinsically scalable GPU-parallel algorithms for complete 4-opt, 5-opt, 6-opt and variable λ -opt ($2 \leq \lambda \leq 6$) search on global TSP tours.
- 3 Design of linear-time complexity methods for detecting non-interacting variable λ -opt moves ($2 \leq \lambda \leq 6$).
- 4 Architectural design enabling seamless deployment across arbitrary numbers of GPUs.

4.1. All valid pure 5-opt and 6-opt reconnection schemes

When removing k edges from an existing TSP tour, the algorithm needs to reconnect k subtours with k new edges while ensuring tour validity. In addition to the well-documented pure 2-/3-/4-opt reconnection schemes, all pure 5-/6-opt reconnection schemes require systematic identification. The fundamental challenge lies in the $2^{k-1} \times (k-1)!$ possible reconnection schemes for k -opt, where manual enumeration quickly becomes infeasible.

This computational requirement is reformulated as a numerical puzzle. Fig. 5 shows that the 5-opt requires reconnecting 10 nodes, while the 6-opt involves 12 nodes. The task therefore reduces to enumerating all connection schemes that:

- Begin at node 1 and terminate at node 10 (5-opt) or 12 (6-opt).
- Maintain a single continuous tour through all nodes.

The task is performed using Algorithm 1, which yields 208 valid pure 5-opt reconnection schemes (Fig. 6). By modifying the algorithm to accommodate 6 edges ($E = 12$) for the case of $(e_o, e_p, e_q, e_j, e_w, e_t)$, 1935 valid pure 6-opt reconnection schemes are identified (Fig. 7).

4.2. Scalable GPU-parallel 4-opt, 5-opt, 6-opt and variable λ -opt on global tour

This section presents GPU-parallel implementations for pure 4-opt, 5-opt, and 6-opt operations, which are subsequently integrated into a unified variable λ -opt framework ($2 \leq \lambda \leq 6$).

4.2.1. GPU-Parallel 4-opt and 5-opt

For a TSP tour with n nodes, a complete 4-opt evaluation requires checking $\frac{n(n-1)(n-2)(n-3)}{24}$ possible 4-edge combinations, while 5-opt requires examining $\frac{n(n-1)(n-2)(n-3)(n-4)}{120}$ possible 5-edge combinations. The

Algorithm 1 The method for generating all possible valid 5-opt reconnection schemes.

```

Require: Number  $E = 10$  for 5-opt  $\mathcal{N}_5^{opqjw}$ .
1:  $o \leftarrow 1$ ;
2: for  $o' = 3; o' < E + 1; o'++$  do
3:   EDGE  $e_1 \leftarrow (o, o')$ ; // first new edge  $e_1$ ;
4:   for  $p = 2; p < E + 1; p++$  do
5:     if  $p! = o \& \& p! = o'$  then
6:       for  $p' = 2; p' < E + 1; p'++$  do
7:         if  $p'! = o \& \& p' = o' \& \& p'! = p$  then
8:           EDGE  $e_2 \leftarrow (p, p')$ ; // second new edge  $e_2$ ;
9:           for  $q = 2; q < E + 1; q++$  do
10:            if  $q! = o \& \& q! = o' \& \& q! = p \& \& q! = p$  then
11:              for  $q' = 2; q' < E + 1; q'++$  do
12:                if  $q'! = o \& \& q' = o' \& \& q'! = p \& \& q'! = q$ 
13:                  then
14:                    EDGE  $e_3 \leftarrow (q, q')$ ; // second new edge
15:                    .... // similar way to loop from 2 to
16:                     $E + 1$  to get edges  $e_4, e_5$ ;
17:                  end if
18:                end for
19:              end if
20:            end for
21:          end if
22:        end for
23:      end for
24: Delete combinations of 5 edges  $(e_1, e_2, e_3, e_4, e_5)$  that cut the original
    TSP tour.
25: for each combination of new 5 edges  $(e_1, e_2, e_3, e_4, e_5)$  do
26:   valid  $\leftarrow$  Start from number 1 of edge  $e_1$ , traverse all other four
    edges  $e_2, e_3, e_4, e_5$ , and end with number 10;
27:   if valide then
28:     This scheme is a valid 5-opt reconnection scheme;
29:   end if
30: end for

```

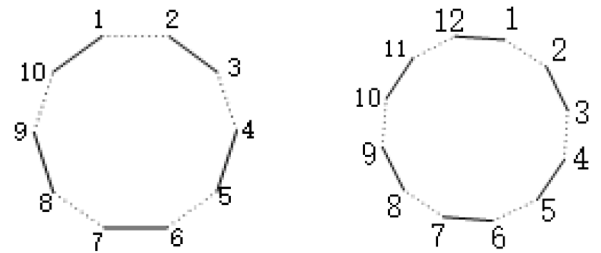


Fig. 5. Removal of 5 or 6 edges from the original tour generates 10 or 12 nodes requiring reconnection.

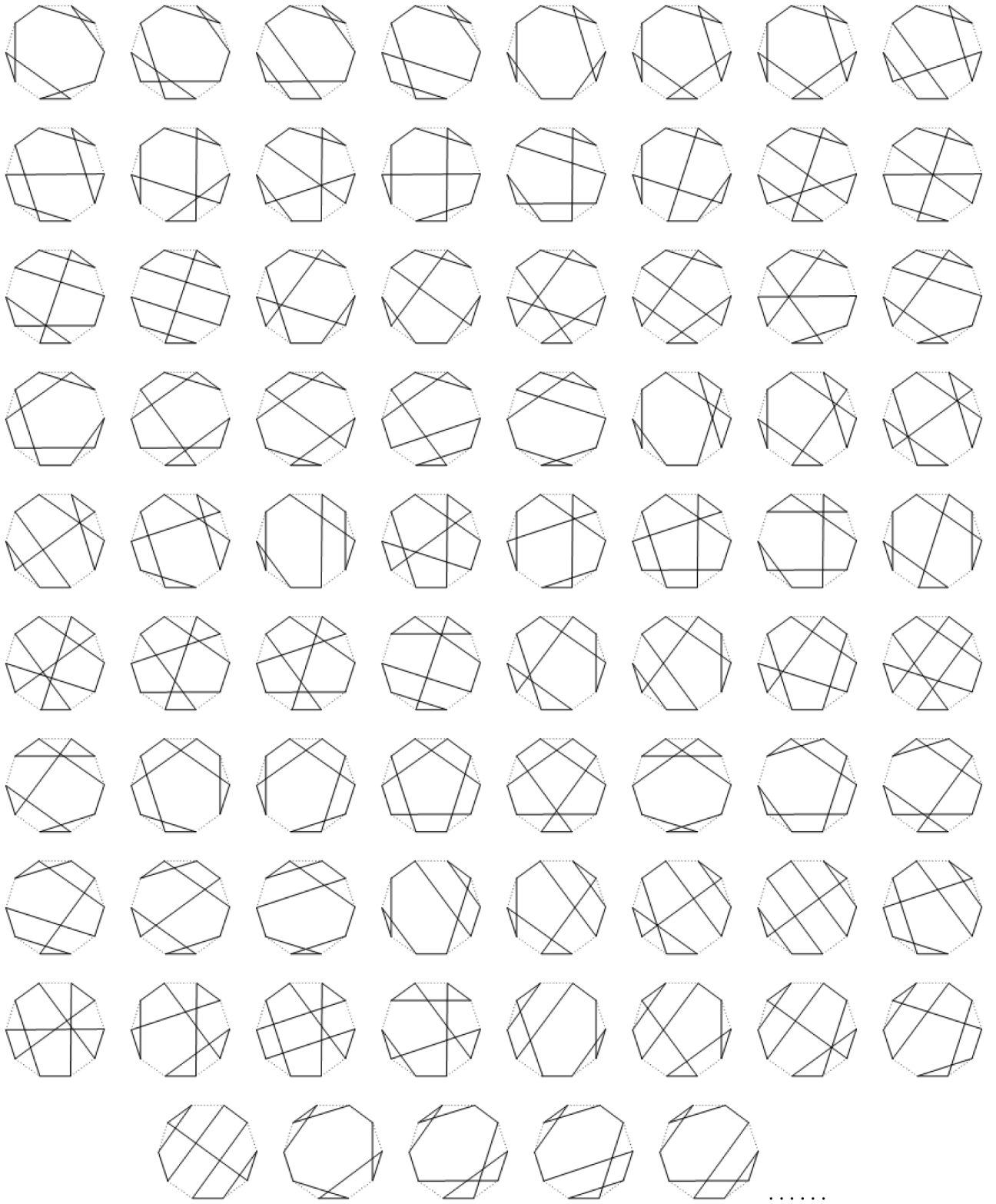


Fig. 6. Seventy-seven representative samples from the total 208 pure 5-opt reconnection schemes.

GPU-parallel 4-/5-opt algorithm aims to execute all these 4-/5-opt checks on the GPU.

Significant computational difficulties emerge when attempting to solve a quartic equation with a unit leading coefficient for the 4-opt case and a quintic equation with a unit leading coefficient for the 5-opt case. Instead of analytically solving these quartic or quintic equations, the

proposed GPU-parallel 4-/5-opt algorithm extends Rocki's GPU-parallel 2-opt algorithm. Its procedure is depicted in Fig. 8.

Assuming that the current TSP tour consists of n nodes and n edges, all the $\frac{n(n-1)}{2}$ possible pairs of edges are distributed as shown in Fig. 8(a, b). These edge-pairs can be mapped to CUDA threads as in Rocki's algorithm.

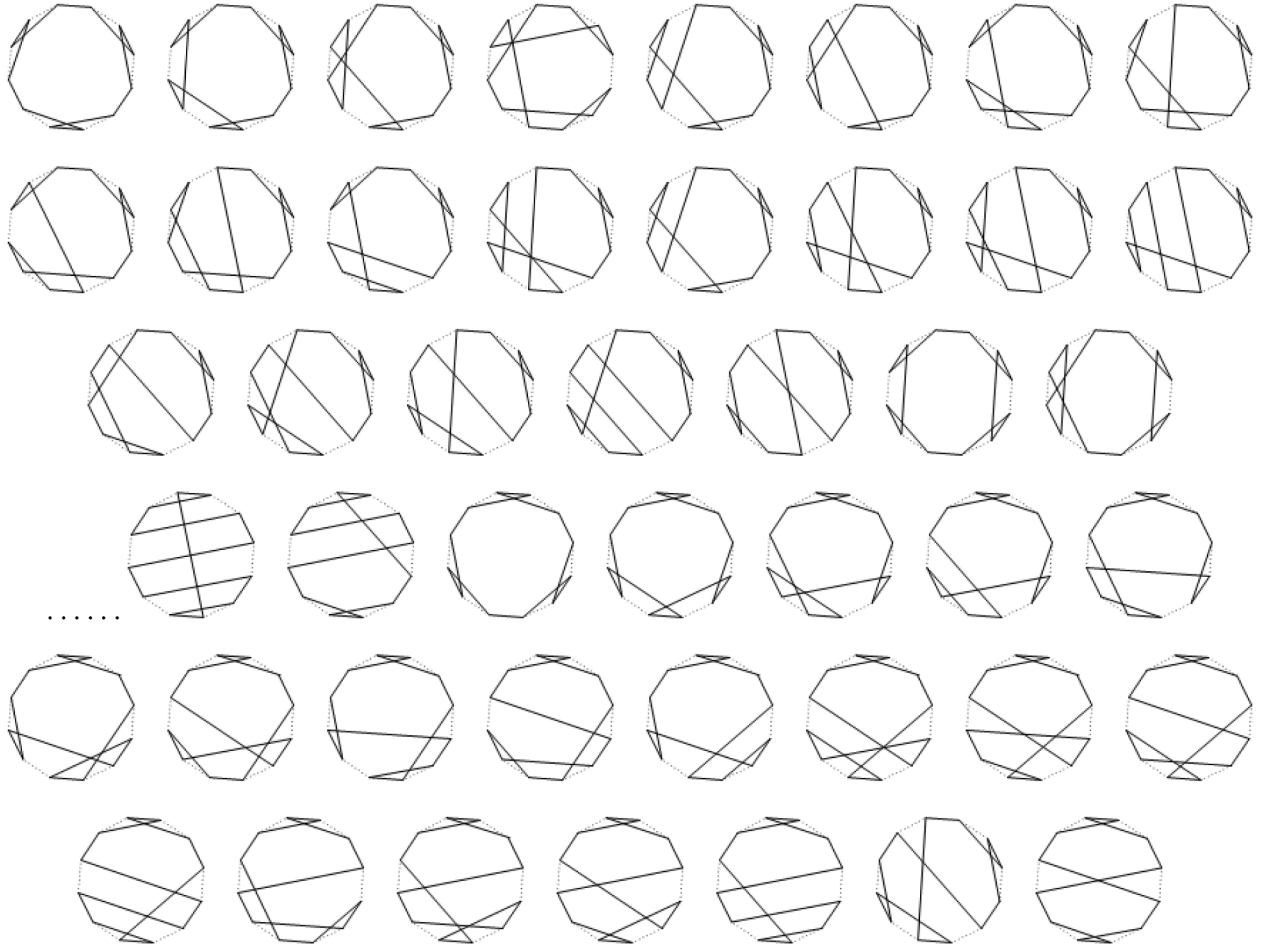


Fig. 7. Forty-three representative samples from the total 1935 pure 6-opt reconnection schemes.

For the 4-opt $\mathcal{N}_4^{opqj}$, N is set to $N = \frac{n(n-1)}{2}$, with all $\frac{N(N-1)}{2}$ indices mapped to CUDA threads, as shown in Fig. 8(c). Each cell in Fig. 8(c) represents an index $id_{(X4,Y4)}$ where $0 \leq id_{(X4,Y4)} \leq \frac{N(N-1)}{2}$. Each $id_{(X4,Y4)}$ can be split into two values $X4$ and $Y4$ where $0 \leq X4, Y4 \leq \frac{n(n-1)}{2}$. A single $X4$ or $Y4$ can be parsed into two indices in Fig. 8(a), which ultimately correspond to four edges along the current tour, as shown in Fig. 8(b). Once a CUDA thread obtains four edges of the current tour, the algorithm searches through all 25 pure 4-opt reconnection schemes shown in Fig. 2 and completes one 4-opt check.

For 5-opt $\mathcal{N}_5^{opqjw}$, the direct extension to 5-opt through recursive mapping (letting $N' = N(N-1)/2$ and allocating $N'(N'-1)/2$ thread indices) would generate excessive computational load, with the potential to accommodate up to 8-opt operations. Instead, the proposed solution implements a streamlined approach that iterates $(n-j)$ times over each valid 4-edge combination within the CUDA kernel (Algorithm 2). Each CUDA thread, upon receiving 5 edges from the current tour, evaluates all 208 pure 5-opt reconnection schemes (Fig. 6) to complete one 5-opt check.

To demonstrate that the proposed GPU-parallel 4-opt, 5-opt methods can encompass all possible $\frac{n(n-1)(n-2)(n-3)}{24}$ four-edge combinations and $\frac{n(n-1)(n-2)(n-3)(n-4)}{120}$ five-edge combinations along the global TSP tour, an example is presented in Fig. 8. The two numbers in each cell of the first column in Fig. 8(c) represent four edges: the first number, 0, denotes two edges, $e_0(\pi_0, \pi_1)$ and $e_1(\pi_1, \pi_2)$; the second number, ranging from 1 to $N = \frac{n(n-1)(n-2)(n-3)}{24}$, represents another two edges, $e_i(\pi_i, \pi_{i+1})$ and $e_j(\pi_j, \pi_{j+1})$, as illustrated in Fig. 8(a and b). This range encompasses all possible two-edge combinations for a 2-opt move. Thus, the first column in Fig. 8(c) enumerates all possible four-edge combinations starting from

the two edges $e_0(\pi_0, \pi_1)$ and $e_1(\pi_1, \pi_2)$. This enumeration strategy is extended to all other columns in Fig. 8(c), ensuring that the method covers all possible four-edge combinations of the global TSP tour. For the 5-opt $\mathcal{N}_5^{opqjw}$, by iterating over j and w where $j < w < n$ for each valid set of four edges, with w serving as the fifth edge, the method accounts for all possible five-edge combinations of the global TSP tour.

4.2.2. GPU parallel 6-opt

Given a TSP tour with n cities, there are a total of $\frac{n(n-1)(n-2)(n-3)(n-4)(n-5)}{720}$ possible 6-edge combinations that need to be verified along the global tour. Thus, it is necessary to develop a GPU-parallel 6-opt algorithm that maps all 6-edge combinations to CUDA threads.

Solving a sextic equation with a unit leading coefficient presents significant challenges. Rather than solving this sextic equation, this work proposes a GPU-parallel 6-opt algorithm that extends Rocki's 3-opt and 2-opt algorithms.

The proposed GPU-parallel 6-opt method is illustrated in Fig. 9. Assuming the current TSP tour comprises n cities and n edges, all possible $\frac{n(n-1)(n-2)}{6}$ sets of 3 edges are systematically enumerated in Fig. 9(a). These sets can be mapped to CUDA threads as in Rocki's 3-opt method.

For 6-opt, N is set to $N = \frac{n(n-1)(n-2)}{6}$, subsequently, all $N(N-1)/2$ combinations are mapped to CUDA threads as depicted in Fig. 9(b). Each cell in Fig. 9(b) represents an index pair $(X6, Y6)$, which ultimately corresponds to six edges along the current TSP tour, as illustrated in Fig. 9(a). When a CUDA thread identifies a valid set of six edges, the algorithm computes all 1935 pure 6-opt reconnection schemes and

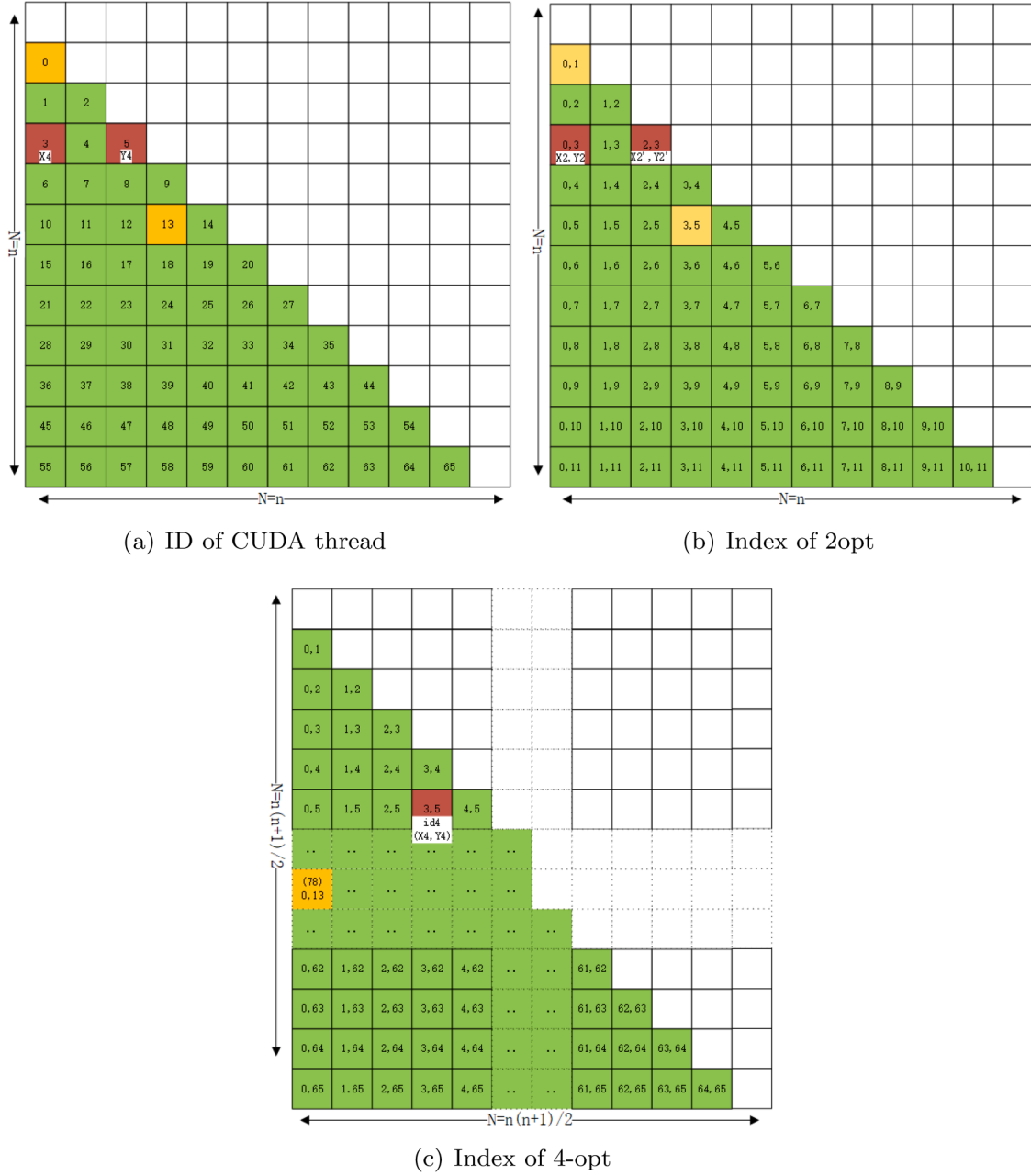


Fig. 8. Thread indexing scheme for GPU-parallel 4-opt: (a) Basic thread ID mapping; (b) 2-opt indexing with $N = n$; (c) 4-opt indexing with $N = n(n-1)/2$.

completes one 6-opt check. The corresponding pseudocode is presented in Algorithm 3.

To demonstrate that the proposed GPU-parallel 6-opt ($\mathcal{N}_6^{opqjw}$) method can encompass all possible $\frac{n(n-1)(n-2)(n-3)(n-4)(n-5)}{720}$ six-edge combinations along the global TSP tour, an example is presented in Fig. 9. The two values in each cell of the first column in Fig. 9(b) correspond to six edges: the first value, 0, represents three edges $e_0(\pi_0, \pi_1)$, $e_1(\pi_0, \pi_1)$, $e_2(\pi_1, \pi_2)$, as illustrated in Fig. 9(a). The second value, ranging from 1 to $N = n(n-1)(n-2)/6$, denotes another three edges $e_i(\pi_i, \pi_{i+1})$, $e_j(\pi_i, \pi_{i+1})$, $e_t(\pi_i, \pi_{i+1})$. This range comprehensively covers all possible three-edge combinations, as shown in Rocki's 3-opt method in Fig. 9(a). Consequently, the first column in Fig. 9(b) can account for all potential six-edge combinations originating from the initial three edges $e_0(\pi_0, \pi_1)$, $e_1(\pi_0, \pi_1)$, $e_2(\pi_1, \pi_2)$. This enumeration pattern is systematically extended to all other columns in Fig. 9(b). Therefore, the proposed method guarantees complete coverage of all possible six-edge combinations along the global TSP tour.

4.2.3. GPU parallel variable 2-/3-/4-/5-/6-opt

The aforementioned GPU-parallel 4-opt, 5-opt, and 6-opt algorithms, together with Rocki's 2-opt and 3-opt GPU-parallel methods, enable pure fixed k -opt ($2 \leq k \leq 6$) operations on the global TSP tour. However, these algorithms do not support GPU-parallel variable λ -opt ($2 \leq \lambda \leq 6$) optimisation. To achieve variable λ -opt, the following two strategies can be employed:

• Iterative application of mixed k -opt until achieving variable λ -optimality

For the same global tour, mixed k -opt evaluation ($2 \leq k \leq 6$) is applied for ascending values of k , followed by the execution of non-interacting variable λ -opt ($2 \leq \lambda \leq 6$) moves. This procedure is iteratively repeated until the variable λ -optimal state is achieved.

• Progressively increasing λ -optimal TSP tours from $\lambda = 2$ to $\lambda = 6$

Beginning with an initial TSP tour, iterative 2-opt is first applied to obtain a 2-optimal tour. Subsequently, iterative λ -opt ($\lambda > 2$)

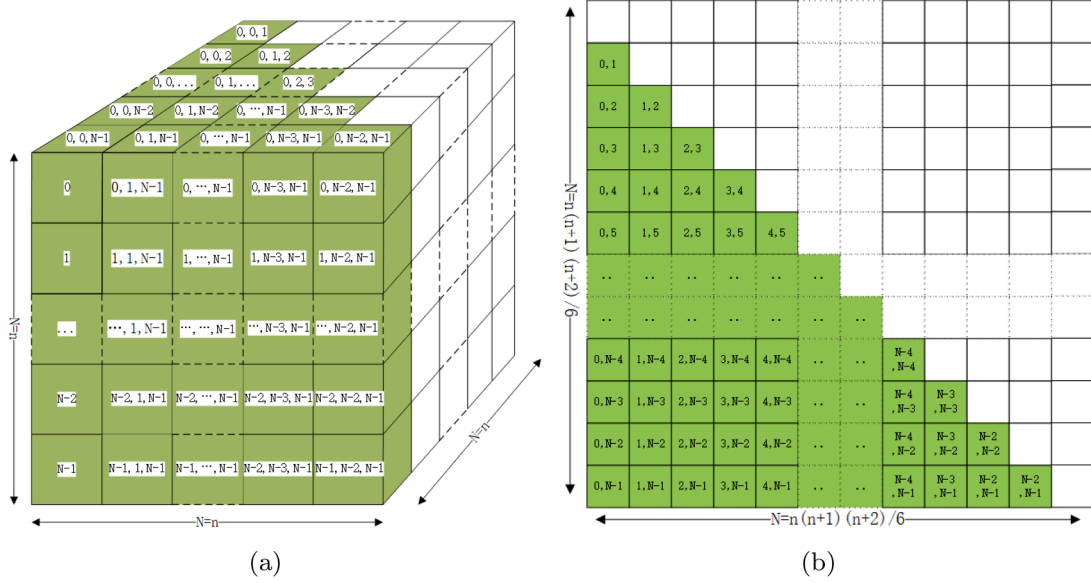


Fig. 9. Indices of GPU-parallel 6-opt. (a) Indices of GPU-parallel 3-opt implementation where $N = n$. (b) Indices of GPU-parallel 2-opt implementation where $N = n(n-1)(n-2)/6$.

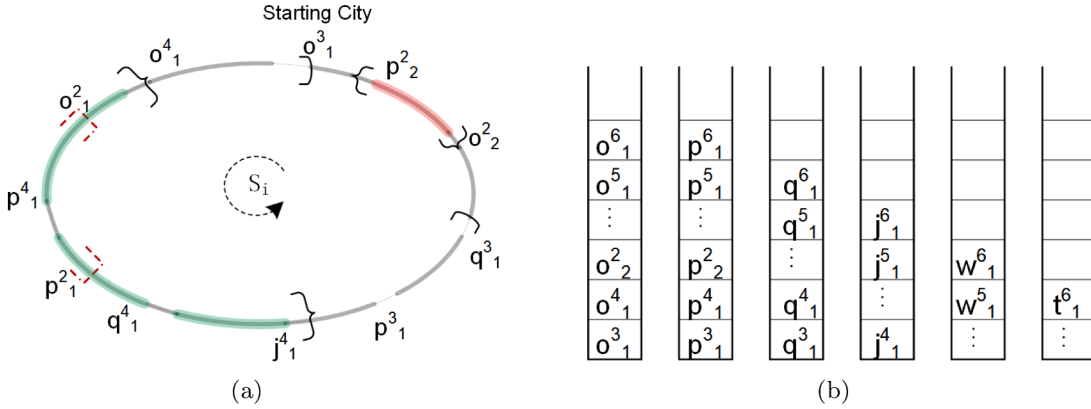


Fig. 10. (a) Criterion for identifying non-interacting variable or fixed 2-/3-/4-/5-/6-opt moves on a TSP tour s_i ; (b) selection of non-interacting 2-/3-/4-/5-/6-opt moves on tour s_i using six stacks.

operations are applied to the resulting $(\lambda - 1)$ -optimal tour until a λ -optimal tour is obtained for $\lambda = 3, 4, 5, 6$.

4.3. Judge non-interacting variable λ -opt

After one iteration of the GPU-parallel fixed or variable λ -opt ($2 \leq \lambda \leq 6$) search on the global TSP tour, thousands of individual valid 2-/3-/4-/5-/6-opt moves may be generated, each independently capable of improving the current tour. However, if executed without proper selection, these moves may disconnect the tour into disjoint segments. To support concurrent execution of multiple 2-/3-/4-/5-/6-opt moves on the same tour, the algorithm needs to identify non-interacting moves with minimal time complexity.

A serial algorithm is proposed for selecting non-interacting variable λ -opt or fixed k -opt ($2 \leq \lambda \leq 6$ and $2 \leq k \leq 6$) moves on the same TSP tour with linear-time complexity. The algorithm consists of two core components: (1) the criterion for non-interacting variable 2-6-opt moves (Section 4.3.1), and (2) the selection algorithm based on this criterion (Section 4.3.2).

4.3.1. Criterion of non-interacting variable λ -opt

The criterion for non-interacting variable 2-/3-/4-/5-/6-opt moves utilises the current TSP tour ordering.

Consider randomly distributed moves on the same TSP tour: 2-opt moves $\mathcal{N}_2^{op}$, 3-opt moves $\mathcal{N}_3^{opq}$, 4-opt moves $\mathcal{N}_4^{opqj}$, 5-opt moves $\mathcal{N}_5^{opqjw}$, 6-opt moves $\mathcal{N}_6^{opqjw}$ where o, p, q, j, w, t denote positions in the TSP tour order ($0 \leq o < p < q < j < w < t < n$), and edges $e_o = (o, o+1)$, $e_p = (p, p+1)$, $e_q = (q, q+1)$, $e_j = (j, j+1)$, $e_w = (w, w+1)$, $e_t = (t, t+1)$ represent the edges to be removed.

The subtour from city o to the respective terminal city p, q, j, w , or t of $\mathcal{N}_2^{op}$, $\mathcal{N}_3^{opq}$, $\mathcal{N}_4^{opqj}$, $\mathcal{N}_5^{opqjw}$, or $\mathcal{N}_6^{opqjw}$ respectively defines a protected range that needs to remain uninterrupted. This protected range establishes the non-interacting criterion: the subtour range must not cross any edges marked for removal. For illustration, Fig. 10(a) shows the complete subtour from city o_1^4 to j_1^4 of $\mathcal{N}_4^{o_1^4 p_1^2 q_1^3 j_1^4}$ entirely contained within the range from o_1^3 to p_1^3 of $\mathcal{N}_3^{o_1^3 p_1^2 q_1^3}$. In this configuration:

- If $\mathcal{N}_3^{o_1^3 p_1^2 q_1^3}$ is selected first, then $\mathcal{N}_4^{o_1^4 p_1^2 q_1^3 j_1^4}$ qualifies as a non-interacting move.
- Conversely, if $\mathcal{N}_4^{o_1^4 p_1^2 q_1^3 j_1^4}$ is selected first, then $\mathcal{N}_3^{o_1^3 p_1^2 q_1^3}$ becomes ineligible.

4.3.2. Algorithm to select non-interacting variable 2-/3-/4-/5-/6-opt moves

Based on the aforementioned non-interacting criterion, an algorithm is introduced to identify compatible variable 2-/3-/4-/5-/6-opt moves

Algorithm 2 GPU-parallel 4-opt and 5-opt.

Require: A TSP tour with n cities. N is set to $N = n(n-1)/2$ and $\text{maxQuantity4optCheck} = N(N-1)/2$ and set CUDA kernel with fixed 1 dimensional BLOCKSIZE and GRIDSIZE.

```

1:  $tid \leftarrow \text{threadIdx.x} + \text{blockDim.x}$ ;
2:  $\_shared\_ \text{tspTour}[n]$ ;
3:  $\_shared\_ \text{4optReconnectionSchemes}[25]$  or  $\text{5optReconnectionSchemes}[208]$ ;
4: Upload entire TSP tour, 25 4-opt or 208 5-opt reconnection schemes to shared memory;
5:  $id \leftarrow tid + \text{startBase}$ ; // one kernel launch takes responsible for a part of searching job.
6: if  $id < \text{maxQuantity4optCheck}$  then
7:    $X4, Y4 \leftarrow \text{function parse}(id)$ ;
8:    $o, p \leftarrow \text{function parse}(X4)$ ;
9:    $q, j \leftarrow \text{function parse}(Y4)$ ;
10:  if  $o < p < q < j$  then
11:    if node  $o|p|q|j$  has not already been occupied for any opt then
12:      if 4-opt then
13:        check all 25 4-opt reconnection schemes for edges
14:         $e_o, e_p, e_q, e_j$ ;
15:        Mark selected 4-opt reconnection  $schemeID$ ; // first or best 4-opt
16:        break;
17:      end if
18:      if 5-opt then
19:        for each  $w > j$  &&  $w < n$  do
20:          if node  $w$  has not already been occupied for any opt then
21:            check all 208 5-opt reconnection schemes for edges  $e_o, e_p, e_q, e_j, e_w$ ;
22:            Mark selected 5-opt reconnection  $schemeID$ ; // first or best 5-opt
23:            break;
24:          end if
25:        end for
26:      end if
27:      if  $schemeID > 0$  then
28:        Store information of the selected 4-opt or 5-opt at city  $\pi_o$  on global memory;
29:        break; // Select the first 4-/5-opt.
30:      end if
31:    end if
32:  end if

```

with single-pass traversal of tour s_i , achieving linear-time complexity. The algorithm employs six stacks (Fig. 10(b)) with the detailed procedure outlined in Algorithm 4.

The core of Algorithm 4 is that the algorithm traverses the TSP tour in the same order as used in the procedure of GPU-parallel 2-/3-/4-/5-/6-opt search illustrated in Section 4.2, and the algorithm only compares each newly met λ -opt move with the last selected λ -opt move stored at the top of the stack. The function **JudgeNonInteracting** in Algorithm 4 judges whether a candidate λ -opt move interacts with the previously selected one based on the non-interacting criterion explained in Section 4.3.1.

4.4. Seamless deployment across arbitrary GPU quantities

Since all GPU-parallel 2-/3-/4-/5-/6-opt checks are independent, inter-GPU communication can be avoided, the proposed parallel λ -opt algorithm can be deployed on any number of GPUs.

Given m GPUs, the proposed method requires dividing the numbers from 0 to N_{total} , the total number of 4-/5-/6-opt checks, into m segments

Algorithm 3 GPU-parallel 6-opt $\mathcal{N}_6^{opqjw}$.

Require: A TSP tour with n cities. N is set to $N = n(n-1)(n-2)/6$ and $\text{maxQuantity6optCheck} = N(N-1)/2$, and set CUDA kernel with fixed 1 dimensional BLOCKSIZE and GRIDSIZE.

```

1:  $tid \leftarrow \text{threadIdx.x} + \text{blockDim.x}$ ;
2:  $\_shared\_ \text{tspTour}[n]$ ;  $\_shared\_ \text{6optReconnectionSchemes}[1935]$ ;
3: Upload entire TSP tour and 1935 6-opt reconnection schemes to shared memory;
4:  $id \leftarrow tid + \text{startBase}$ ; // computed by max threads of one kernel launch and GRIDSIZE.
5: if  $id < \text{maxQuantity6optCheck}$  then
6:    $X6, Y6 \leftarrow \text{function parse}(id)$ ;
7:    $o, p, q \leftarrow \text{function parse}(X6)$ ;
8:    $j, w, t \leftarrow \text{function parse}(Y6)$ ;
9:   if  $o < p < q < j < w < t$  then
10:    Compute  $oldLength$  of edges  $(e_o, e_p, e_q, e_j, e_w, e_t)$ ;
11:    for each  $schemeID$  in 1935 6-opt reconnection schemes do
12:      Compute  $newLength$  of  $(e'_o, e'_p, e'_q, e'_j, e'_w, e'_t)$ ;
13:      if  $newLength < oldLength$  then
14:        Mark  $schemeID$ ; // first or best 6-opt
15:      end if
16:    end for
17:    if  $schemeID > 0$  then
18:      Store information of the selected 6-opt at city  $\pi_o$  on global memory;
19:      break; // Select the first 6-opt.
20:    end if
21:  end if
22: end if

```

Algorithm 4 Sequential selection of non-interacting k -opt moves on s_i .

Require: From s_i 's starting city π_o , traverse each city π_o according to current tour order of s_i . Consider a candidate $\mathcal{N}_2^{op}$ or $\mathcal{N}_3^{opq}$ or $\mathcal{N}_4^{opqj}$ or $\mathcal{N}_5^{opqjw}$ or $\mathcal{N}_6^{opqjw}$, $o < p < q < j < w < t$, $p \neq (o+1)$, $q \neq (p+1)$, $j \neq (q+1)$, $w \neq (j+1)$, $t \neq (w+1)$, stack A is for city π_o , stack B is for city π_p , stack C is for city π_q , similar with stacks D, E, F are for city π_j, π_w, π_t respectively.

```

1: for each city  $\pi_o$  from the starting city  $\pi_o$  do
2:   if  $A.size > 0$  &&  $\pi_o$  is in Stack B/C/D/E/F then // (City  $\pi_o$  has been occupied by previously selected  $k$ -opt move.)
3:     Mark  $A.top$  possessing a selected non-interacting  $k$ -opt move;
4:     From stack  $A$  to  $F$ , only pop  $k$  stacks' top elements.
5:   else if  $\pi_o$  possesses a  $\mathcal{N}_2^{op}$  or  $\mathcal{N}_3^{opq}$  or  $\mathcal{N}_4^{opqj}$  or  $\mathcal{N}_5^{opqjw}$  or  $\mathcal{N}_6^{opqjw}$  then
6:     Extract  $\pi_{p|q|j|w|t}$  from  $\pi_o$ 's information package;
7:     if  $Stacks A.size == 0$  then
8:       Mark first selected  $k$ -opt move;
9:       Push  $\pi_{o|p|q|j|w|t}$  to stacks  $A|B|C|D|E|F$  respectively;
10:      Mark  $\pi_{p|q|j|w|t}$  is in stack  $B|C|D|E|F$ ;
11:    else JudgeNonInteracting // (Compare the new  $k$ -opt with the last selected variable  $\lambda$ -opt move on top of these stacks.)
12:      if Non-interacting then
13:        Push  $\pi_{o|p|q|j|w|t}$  to stacks  $A|B|C|D|E|F$  respectively;
14:        Mark  $\pi_{p|q|j|w|t}$  is in stack  $B|C|D|E|F$ ;
15:      end if
16:    end if
17:  else
18:    Continue next city  $\pi_{(o+1)}$  along  $s_i$ ; // (One edge only participate in one 2-opt move.)
19:  end if
20: end for

```

to balance each GPU's workload. A general task division method is presented in Algorithm 5 where each GPU alternately executes the same amount of these numbers. Furthermore, considering that a CUDA kernel cannot launch an infinite number of threads, the maximum k -edge combinations one kernel launch can execute are fixed as 'maxQuantityKoptCheckDivided' in Algorithm 5.

When using multiple GPUs, results from different GPUs need to be merged. Additionally, some k -opt searching computations are redundant because each city can participate in at most one k -opt move, and no inter-GPU communication is employed. However, this redundancy becomes negligible when the number of GPUs is sufficiently large.

Algorithm 5 A general task division method for parallel k -opt search across m GPUs.

Require: A TSP tour with n cities.

```

1: maxQuantityKoptCheck: represent the total numbers of all  $k$ -opt
   checks;
2: maxQuantityKoptCheckDivided: represent the max checks done by
   one kernel launch;
3: packSize = BLOCKSIZE * GRIDSIZE
4: checksOfOneThread  $\leftarrow$  maxQuantityKoptCheckDivided / packSize
   ;
5: maxStartBase  $\leftarrow$  maxQuantityKoptCheck / maxQuantityKoptCheck
   Divided;
6: if maxStartBase  $\leq$  1 then
7:   cudaSetDevice(0);
8:   kernel_parallel_Kopt(TSP, checksOfOneThread);
9: else
10:  for int startBase = 0; startBase < maxStartBase; startBase += m
     do
11:    cudaSetDevice(0)
12:    kernel_parallel_Kopt(TSP, checksOfOneThread, startBase);
13:    cudaSetDevice(1)
14:    kernel_parallel_Kopt(TSP, checksOfOneThread, startBase
       + 1);
15:    ...
16:    cudaSetDevice(m)
17:    kernel_parallel_Kopt(TSP, checksOfOneThread, startBase +
       m - 1);
18:  end for
19: end if

```

5. Experiments

Experimental comparisons were conducted as follows: First, the performance of the proposed parallel 4-opt, 5-opt, 6-opt, and variable λ -opt algorithms was evaluated on a single-GPU in an iterative mode (Section 5.1). Second, a comparison was performed with the other two GPU parallel pure k -opt algorithms, to present the balance between running time and TSP tour quality obtained (Section 5.2). Third, multi-GPU scalability of the proposed GPU-parallel 4-opt was evaluated on a 2-GPU system (Section 5.3).

Experiments were conducted on 12 national TSP instances from TSPLIB (Reinelt, 1991) which represent arbitrary city distributions, with instance sizes reaching up to 33,708 cities. One iteration means each edge of the current TSP tour has been examined to search for possible k -opt optimisation on the global tour. One test commences with the same initial TSP solution and terminates when no further optimisation can be achieved within two consecutive iterations.

The quality of TSP solutions is quantified using two metrics: (1) The percentage deviation of mean results from the optimum, denoted as %PDM (percentage deviation of the mean); and (2) The percentage deviation between the solution obtained and the optimum, referred to as %PD (percentage deviation).

Algorithm 6 Serial k -opt search (first/best) with multiple moves: $k = 2/3/4/5/6$.

Require: A TSP tour with n cities.

```

1: for  $o = 0; o < n; o++$  do
2:   for  $p = o + 1; p < n; p++$  do // 2-opt
3:     for  $q = p + 1; q < n; q++$  do // 3-opt
4:       for  $j = q + 1; j < n; j++$  do // 4-opt
5:         for  $w = j + 1; w < n; w++$  do // 5-opt
6:           for  $t = w + 1; t < n; t++$  do // 6-opt
7:             oldLength = distance(e(o, o + 1) + e(p, p + 1)
               + ...);
8:             newLengths = distance(reconnection schemes of
               k - opt);
9:             if Select the 'first' optimisation then
10:              if oldLength > newLength then
11:                Mark candidate  $k$ -opt for node  $o$ ;
12:                break and begin next node  $o \leftarrow i + 1$ ;
13:              end if
14:            end if
15:            if Select the 'best' optimisation then
16:              optimisation = oldLength - newLength;
17:              if new_optimisation > old_optimisation then
18:                Mark candidate  $k$ -opt for node  $o$ ;
19:              end if
20:            end if
21:          end for
22:        end for
23:      end for
24:    end for
25:  end for
26: end for
27: for Node  $o$  with candidate  $k - opt$ ;  $o++$  do
28:   Select non-interacting  $k - opt$ ;
29: end for

```

The measured execution time of GPU-parallel algorithms accounts for three components: (1) serial processing operations, (2) parallel computation on the GPU, and (3) essential data transfers between CPU and GPU memory. For robust statistical analysis, seven repeated trials per test were conducted under identical experimental conditions, and the mean computational duration is reported.

Source code of the proposed algorithms has been put on GitHub¹. All algorithms are implemented in C/C++ and tested on two computers: the first has a 12th Gen Intel(R) Core(TM) i7-12700H 2.3 GHz and runs only serial algorithms; the second has two Intel(R) Xeon(R) CPU E5-2680 v4 processors with two NVIDIA RTX A4000 GPUs and CUDA toolkit v12.6 with WDDM mode. The CUDA kernel configuration is set as <<<1024, 1048576>>>.

5.1. Evaluation of the proposed algorithms using iterative mode on a single GPU

In the proposed algorithms, the computational burden primarily lies in the 4-/5-/6-opt search stages, as the selection of non-interacting λ -opt moves maintains an $\mathcal{O}(n)$ time complexity. Experiments show that the proposed GPU-parallel 4-/5-/6-opt search time is influenced by the following factors:

- The limit of GPU on-chip shared memory and the efficiency of code implementation in utilising GPU on-chip shared memory.
- The quality of the initial TSP tour and the rules for selecting the 'first' or 'best' k -opt optimisation for an edge.

¹ https://github.com/wenbaoQIAO/Multiple_GPUs_Parallel_4opt_5opt_6opt_Massive_variable_23456-opt_moves_for_TravelingSalesmanProblem.git

Algorithm 7 Serial mixed λ -opt search (first) with multiple variable λ -opt moves.

Require: A TSP tour with n cities.

```

1: for  $o = 0; o < n; o++$  do
2:   for  $p = o + 2; p < n; p++$  do
3:     Try 2-opt between edges  $e(o, o + 1), e(p, p + 1)$ ;
4:     if Exist 2-opt then
5:       break inner loops and begin next node  $o \leftarrow o + 1$ ;
6:     end if
7:     for  $q = p + 2; q < n; q++$  do
8:       Try 3-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1)$ ;
9:       if Exist 3-opt then
10:        break inner loops and begin next node  $o \leftarrow o + 1$ ;
11:       end if
12:       for  $j = q + 2; j < n; j++$  do
13:        Try 4-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1), e(j, j + 1)$ ;
14:        if Exist 4-opt then
15:          break inner loops and begin next node  $o \leftarrow o + 1$ ;
16:        end if
17:        for  $w = j + 2; w < n; w++$  do
18:          Try 5-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1), e(j, j + 1), e(w, w + 1)$ ;
19:          if Exist 5-opt then
20:            break inner loops and begin next node  $o \leftarrow o + 1$ ;
21:          end if
22:          for  $t = w + 2; t < n; t++$  do
23:            Try 6-opt among edges  $e(o, o + 1), e(p, p + 1), \dots, e(w, w + 1), e(t, t + 1)$ ;
24:            if Exist 6-opt then
25:              break inner loops and begin next node  $o \leftarrow o + 1$ ;
26:            end if
27:          end for
28:        end for
29:      end for
30:    end for
31:  end for
32: end for
33: for Node  $i$  with candidate  $k - opt; i++$  do
34:   Select non-interacting variable 2-/3-/4-/5-/6-opt;
35: end for

```

- The amount of computation per CUDA thread and data structures influencing CUDA computation.

These factors also interact with one another. For example, even when the on-chip shared memory of the GPU is sufficient to store all intermediate variables, using shared memory does not necessarily result in a shorter runtime for obtaining a k -optimal TSP solution compared to avoiding its use. Consider Algorithm 2 as an example: if the ‘first’ strategy is adopted, where once a city π_i is selected by the first candidate 4-5-opt move, all subsequent 4-/5-edge combinations involving π_i are skipped. If the occupancy status of each city is recorded in shared memory, this status is visible only to threads within the same thread block. Consequently, threads in other blocks may perform redundant computations.

Considering these factors and the aim of this study to minimise the time required to achieve a λ -optimal TSP solution, the proposed algorithms are integrated into the following iterative frameworks to compare the algorithms’ efficiency in obtaining λ -optimal TSP solutions. All experiments adopt two strategies at the λ -opt search stage: choosing either the ‘first’ λ -opt or the ‘best’ λ -opt improvement for an edge.

Algorithm 8 Serial mixed λ -opt search (best) with multiple variable λ -opt moves.

Require: A TSP tour with n cities.

```

1: for  $o = 0; o < n; o++$  do
2:   for  $p = o + 2; p < n; p++$  do
3:     Try 2-opt between edges  $e(o, o + 1), e(p, p + 1)$ ;
4:     if Exist 2-opt && (new_optimisation > old_optimisation) then
5:       Mark optimisation;
6:     end if
7:     for  $q = p + 2; q < n; q++$  do
8:       Try 3-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1)$ ;
9:       if Exist 3-opt && (new_optimisation > old_optimisation) then
10:        Mark optimisation;
11:       end if
12:       for  $j = q + 2; j < n; j++$  do
13:        Try 4-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1), e(j, j + 1)$ ;
14:        if Exist 4-opt && (new_optimisation > old_optimisation) then
15:          Mark optimisation;
16:        end if
17:        for  $w = j + 2; w < n; w++$  do
18:          Try 5-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1), e(j, j + 1), e(w, w + 1)$ ;
19:          if Exist 5-opt && (new_optimisation > old_optimisation) then
20:            Mark optimisation;
21:          end if
22:          for  $t = w + 2; t < n; t++$  do
23:            Try 6-opt among edges  $e(o, o + 1), e(p, p + 1), \dots, e(w, w + 1), e(t, t + 1)$ ;
24:            if Exist 6-opt && (new_optimisation > old_optimisation) then
25:              Mark optimisation;
26:            end if
27:          end for
28:        end for
29:      end for
30:    end for
31:  end for
32: end for
33: for Node  $i$  with candidate  $k - opt; i++$  do
34:   Select non-interacting variable 2-/3-/4-/5-/6-opt;
35: end for

```

- Iterative Serial/Parallel k -opt Search (first/best) with Multiple Moves: One iteration of the serial version contains k loops through the n edges to find each edge’s first or best k -opt optimisation on the global tour (Algorithm 6). One iteration of the parallel version adopts GPU-parallel fixed k -opt proposed in this paper (Algorithms 2 and 3). Both versions conclude with the selection and execution of multiple non-interacting fixed k -opt moves. The iteration continues until a k -optimal solution is achieved.
- Iterative Serial/Parallel Mixed λ -opt Search (first/best) with Multiple Variable λ -opt Moves: One iteration of the serial version attempts to find each edge’s first (Algorithm 7) or best (Algorithm 8) λ -opt move in the current λ -loop; if no λ -opt move is found for the edge, the algorithm progresses to find $(\lambda + 1)$ -opt in the next loop for this edge. One iteration of the parallel version attempts to find mixed 2- to λ -opt opportunities by assigning one CUDA thread to the parallel 2- to λ -opt methods (Algorithms 2 and 3). Both versions conclude with the selection and execution of multiple non-interacting variable

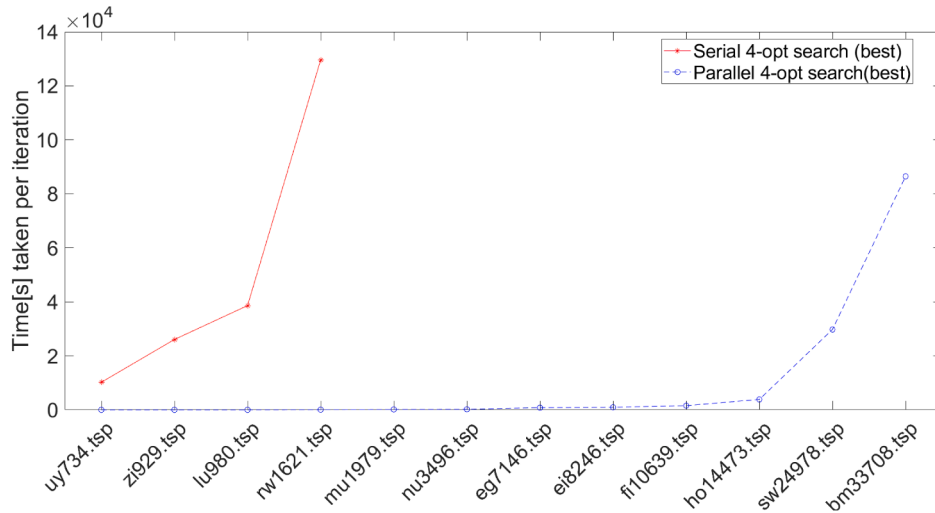


Fig. 11. Comparison of the running time per iteration for serial and parallel complete 4-opt search using the ‘best’ improvement strategy (Serial Algorithm 6 vs. Parallel Algorithm 2). In the serial version, one iteration that exceeded 24 h was not precisely timed.

λ -opt moves. The iteration continues until a λ -optimal TSP solution is achieved.

- Serial/Parallel Iterative Increasing λ -optimal (first/best): Using the above ‘Iterative Serial/Parallel k -opt Search (first/best) with Multiple Moves’ algorithm to obtain a λ -optimal solution, starting from $k = 2$ and ending at $k = \lambda$.
- Multi-GPU Parallel Iterative Increasing λ -optimal (first/best): Deploying the above ‘Parallel Iterative Increasing λ -optimal (first/best)’ algorithm on m GPUs.

5.1.1. Evaluation of parallel fixed 4-/5-/6-opt on single GPU

Given the same initial TSP tour, both serial and parallel ‘iterative fixed k -opt search with multiple k -opt moves’ algorithms are tested. Experimental results are recorded in Table 1, Figs. 11–13.

When considering only a single iteration of complete 4-opt search on the global TSP tour, namely the ‘ k -opt search with the “best” optimisation strategy’, a single GPU iteration completes in 7.5 s for uy734, 10.5 s for zi929, 11.9 s for lu980, 62.1 s for rw1621, 135.1 s for mu1979, 178.3 s for nu3496, 799.3 s for eg7146, 951.6 s for ei8246, 1539.9 s for fi10639, 3845.3 s for ho14473, 29758.3 s for sw24978 and 86457.3 s for bm33708. In contrast, the serial complete 4-opt search takes 10204.4 s for uy734.tsp, 26056.5 s for zi929, 38618.2 s for lu980 and more than 48 h for mu1979 and larger TSP instances (Fig. 11).

When considering the iterative mode of k -opt search and for optimising an initial brute-force TSP tour, the ‘serial fixed k -opt search (first) with multiple k -opt moves’ converges faster than all parallel GPU versions. This convergence continues until the tour is near-optimised, as shown in Figs. 12 and 13. The underlying reason is that k -opt search with the ‘first’ optimisation strategy is a pruning method of the complete k -opt search (version with ‘best’ optimisation strategy), and only performs a near-complete k -opt search on near-optimised TSP tours. Although the GPU-parallel k -opt with ‘first’ strategy also prunes the complete k -opt search range, its difference from the ‘best’ strategy is obvious for large TSP instances like nu3496 as shown in Fig. 13 for the 4-opt test.

One characteristic of the GPU-parallel k -opt search (first or best) algorithms is that these algorithms exhibit nearly constant computation time per iteration, as shown in Figs. 12 and 13.

Regarding the fixed 5-opt and 6-opt algorithms, since the number of all possible 5- or 6-edge exchange combinations is extremely large and each combination checks all 5- or 6-opt reconnection schemes, these procedures require prohibitively long computation times, as evidenced

in Table 1. Although the serial iterative fixed 5-/6-opt with ‘first’ improvement strategy runs relatively faster in the initial iterations, this advantage is insignificant. The significance of the proposed GPU-parallel 5-opt and 6-opt algorithms lies in the fact that, by using an adequate number of GPUs, the running time of complete 5-opt and 6-opt search can be reduced.

From the above comparison, the proposed GPU-parallel 4-opt, 5-opt, 6-opt search (first/best) algorithms are recommended for optimising near-optimal TSP tours rather than brute-force initial TSP tours.

5.1.2. Evaluation of parallel variable λ -opt on single GPU

Two approaches exist to implement variable λ -opt. Given the same brute-force initial TSP tour, both serial and parallel versions of ‘iterative mixed λ -opt search (first/best) with multiple variable λ -opt moves’ and ‘iterative increasing λ -optimal (first/best)’ (Algorithms 7 and 8, and algorithms described in Section 4.2.3) were tested. Experimental results are recorded in Figs. 14–16.

When considering only a single iteration of the complete ‘mixed 2-/3-/4-opt’ search on the global TSP tour (namely the serial and parallel ‘mixed 2-/3-/4-opt search with the “best” improvement strategy’), a single GPU iteration completes in 6.5 s for uy734, 9.8 s for zi929, 11.2 s for lu980, 57.8 s for rw1621, 152.1 s for mu1979, 652.2 s for nu3496, 909.3 s for eg7146, 1103.4 s for ei8246, 1851.9 s for fi10639, 4600.3 s for ho14473, 31692.7 s for sw24978, 98386.5 s for bm33708, whereas the serial complete “mixed 2-/3-/4-opt” search takes 10506.7 s for uy734, 25986.6 s for zi929, 38325.9 s for lu980 and later more than 48 h for mu1979 and larger TSP instances, as shown in Fig. 14. This approach performs similarly to the “fixed 4-opt search with the “best” strategy”, as the maximum number of mixed 2/3/4 edge combinations is the same as that of the fixed 4-opt.

When considering the iterative mode of these two variable λ -opt strategies and for optimising an initial brute-force TSP tour, GPU-parallel “increasing λ -optimal” converges faster than all serial counterparts (Fig. 15). By contrast, the GPU-parallel “mixed λ -opt” converges more slowly than the serial version with the “first” optimisation strategy (Fig. 16). This convergence continues until the tour is near-optimised, mirroring the same phenomenon as the “iterative fixed 4-opt” discussed in Section 5.1.1.

From the above comparison, the proposed GPU-parallel “mixed 2-/3-/4-opt search (first/best)” algorithm is also recommended for optimising near-optimal TSP tours. By contrast, the “parallel iterative increasing λ -optimal” is recommended to optimise a brute-force initial TSP tour.

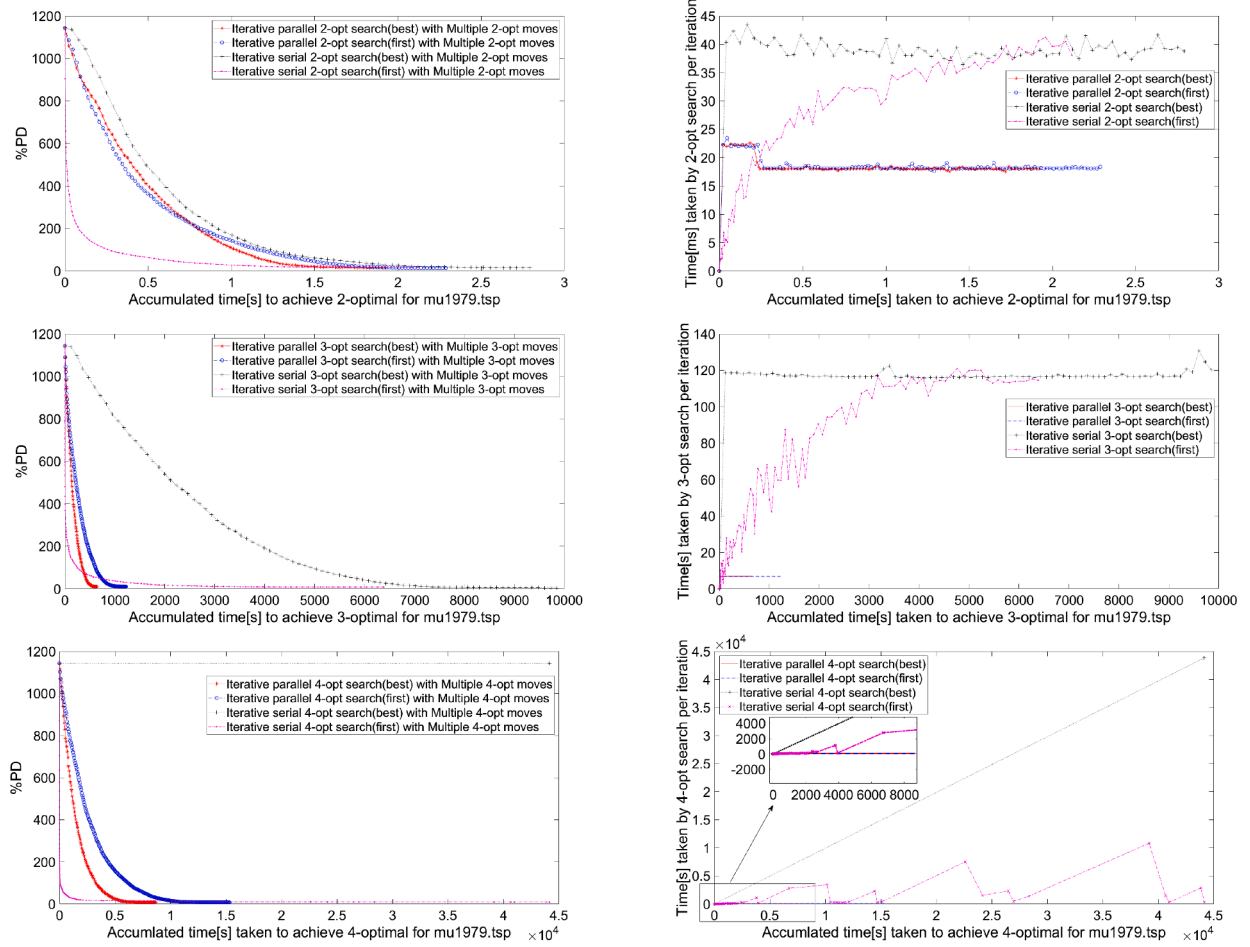


Fig. 12. Convergence dynamics for achieving fixed k -optimal ($k = 2, 3, 4$) solutions using ‘Iterative Serial/Parallel k -opt Search (first/best) with Multiple Moves’ for mu1979.tsp. Left panels: solution quality (PD) versus computation time. Right panels: k -opt search duration per iteration.

Table 1

The overall computation time comparison to achieve k -optimal TSP tour for mu1979.tsp.

PD \Time	Iterative serial Fixed k -opt(first)	Iterative serial Fixed k -opt(best)	Iterative parallel Fixed k -opt(first)	Iterative parallel Fixed k -opt (best)
initial PD	1143.63 \0	1143.63 \0	1143.63 \0	1143.63 \0
2-optimal	16.882 \2.011 s	15.487 \2.915 s	15.988 \2.2413s	16.082 \2.001 s
3-optimal	7.626 \5571.39 s	6.7573 \9610.945s	10.899 \1222.76s	9.19 \665.234 s
4-optimal	7.13 \>56.742 h	1143.63 \>48h	7.055 \4.211 h	6.63 \2.405 h
5-optimal	20.396 \> 60.721 h	1143.63 \>48h	1053.28 \>21.503 h	1013.98 \>32 h
6-optimal	81.749 \>37.053 h	1143.63 \>48h	1099.92 \>22.861 h	1100.68 \>13.84 h
PD \Time	serial increasing k -optimal(first)	serial increasing k -optimal(best)	parallel increasing k -optimal(first)	parallel increasing k -optimal (best)
initial PD	1143.63 \0	1143.63 \0	1143.63 \0	1143.63 \0
2-optimal	15.2726 \2.144 s	14.625 \3.216 s	16.406 \2.299 s	14.575 \1.933 s
3-optimal	7.413 \3464.42 s	9.243 \1370 s	10.078 \354.87 s	9.2684 \166.951 s
4-optimal	7.413 \>48 h	9.243 \>48 h	7.666 \1.605 h	ul 6.4158 \0.8312 h
5-optimal	none \>72 h	none \>72 h	7.255 \>17.66 h	5.9253 \>37.531 h
6-optimal	none \>72 h	none \>72 h	6.957 \>21.35 h	5.6551 \>45.673 h
PD \Time	serial variable mixed k -optimal(first)	serial variable mixed k -optimal(best)	parallel variable mixed k -optimal(first)	parallel variable mixed k -optimal(best)
initial PD	1143.63 \0	1143.63 \0	1143.63 \0	1143.63 \0
2-optimal	none	none	none	none
3-optimal	9.356 \3398.98s	6.166 \24920.8s	10.525 \1087.635s	10.691 \654.190 s
4-optimal	87.869 \33.39 h	1143.63 \>48 h	7.895 \3.282 h	6.383 \2.077 h
5-optimal	809.393 \>48 h	1143.63 \>48 h	30.776 \>22.35 h	1146.63 \>24 h
6-optimal	953.539 \>63 h	1143.63 \>48 h	40.546 \>23.2 h	1143.63 \>24 h

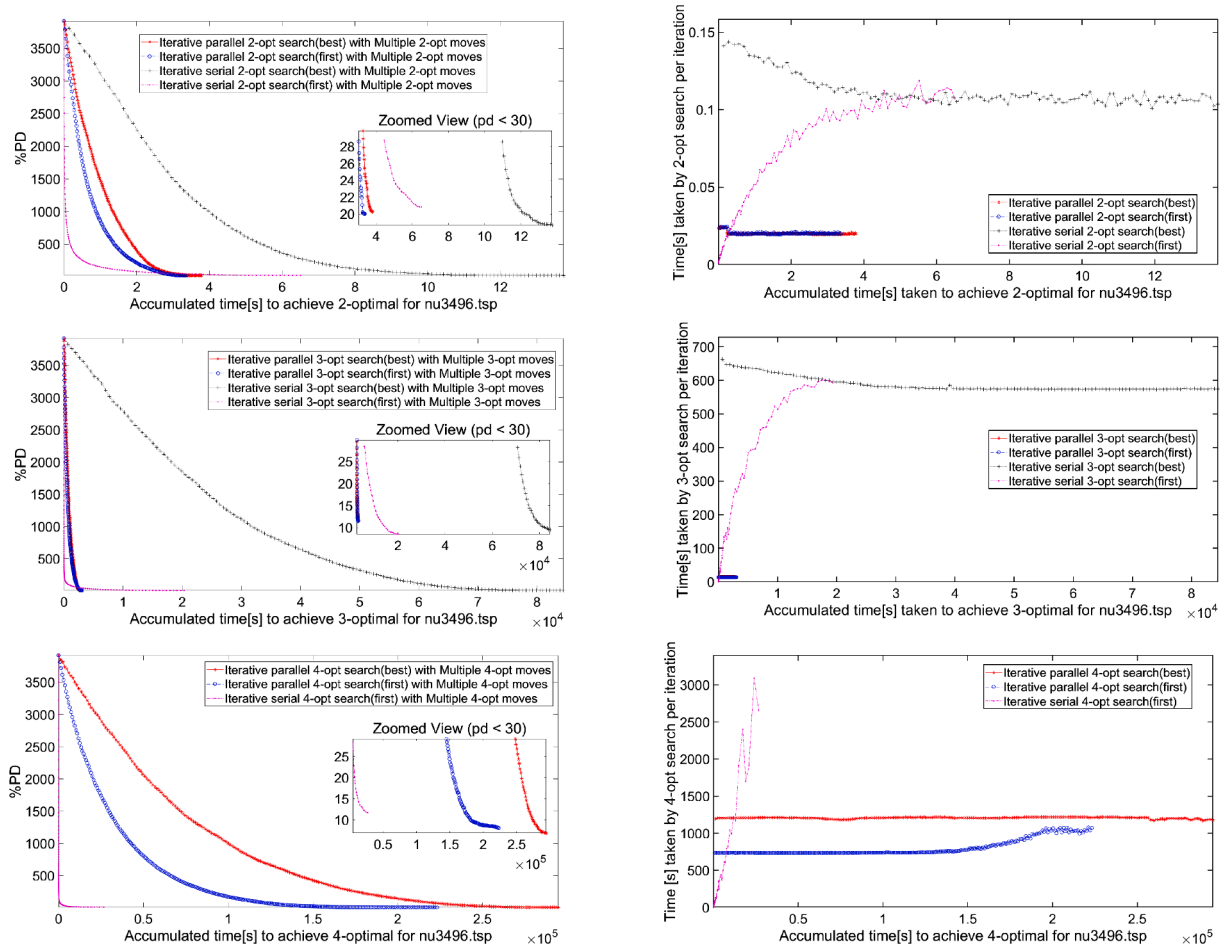


Fig. 13. Convergence dynamics for achieving fixed k -optimal ($k = 2, 3, 4$) solutions using 'Iterative Serial/Parallel k -opt Search (first/best) with Multiple Moves' for nu3496.tsp. Left panels: solution quality (PD) versus computation time. Right panels: k -opt search duration per iteration.

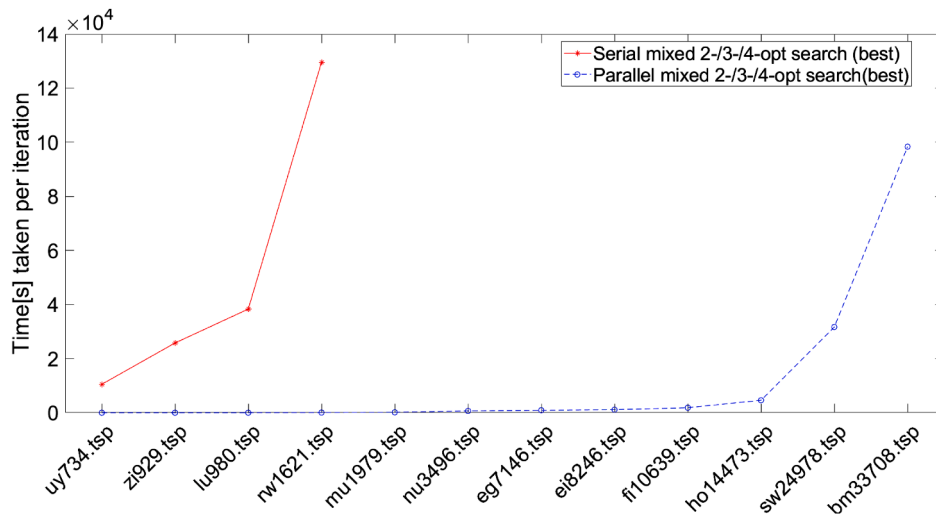


Fig. 14. Comparison of running time per iteration for serial and parallel complete "mixed 2-/3-/4-opt search using the "best" improvement strategy" (Serial Algorithm 7 vs. its parallel version explained in Section 4.2.3). In the serial version, one iteration exceeding 24 hours was not precisely timed.

The "Parallel Iterative Increasing λ -optimal (best)" achieves a 57-fold speedup (0.8312 h vs. >48 h) over its serial counterpart for 1979.tsp, as shown in Table 1. Moreover, the parallel algorithm further enables 5-opt and 6-opt refinements on the 4-optimal tour within feasible computation times.

5.2. Comparative benchmarking against state-of-the-art algorithms

This research focuses on multiple-GPU-parallel complete 4-opt, 5-opt, 6-opt and variable λ -opt search with hybrid variable λ -opt moves on the global TSP tour, in which the search operation depends solely

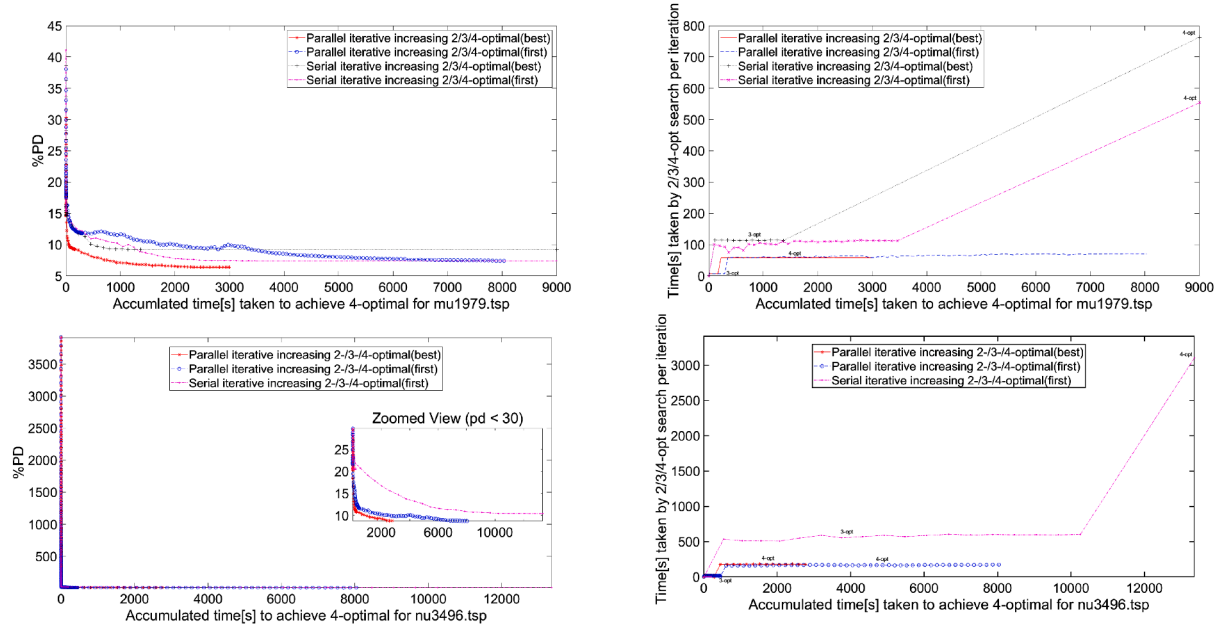


Fig. 15. Convergence dynamics for achieving λ -optimal solutions using “Serial/Parallel Iterative Increasing λ -optimal (first/best)” algorithm for the mu1979.tsp and nu3496.tsp: (left) solution quality (PD) versus computation time; (right) λ -opt search duration per iteration.

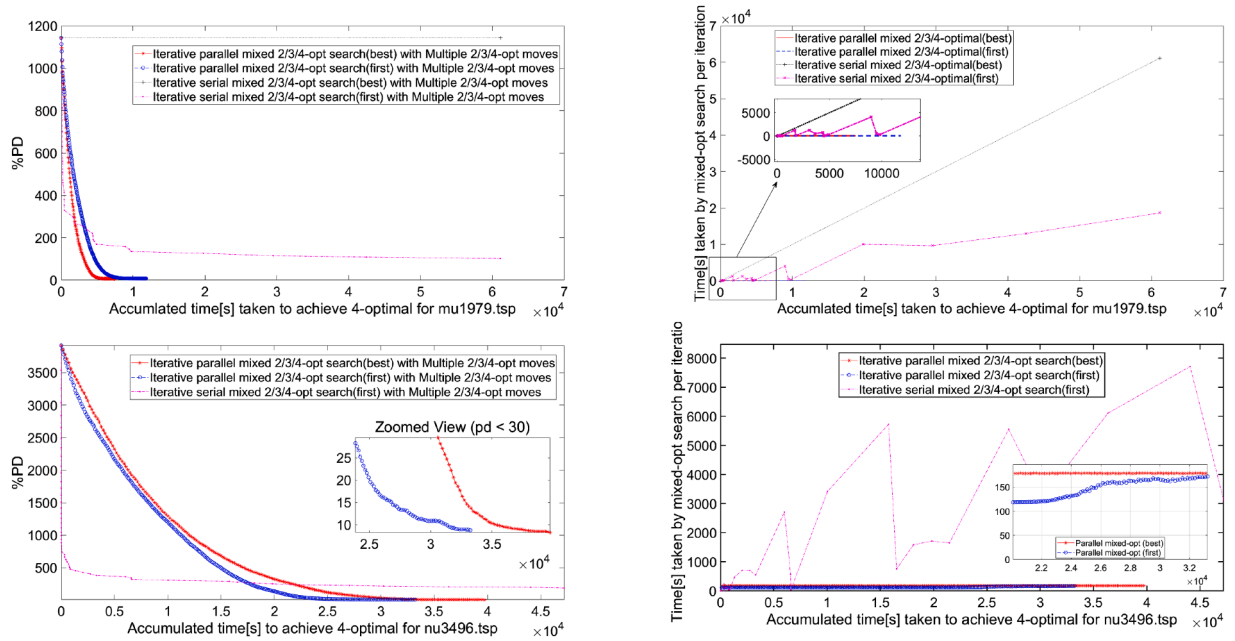


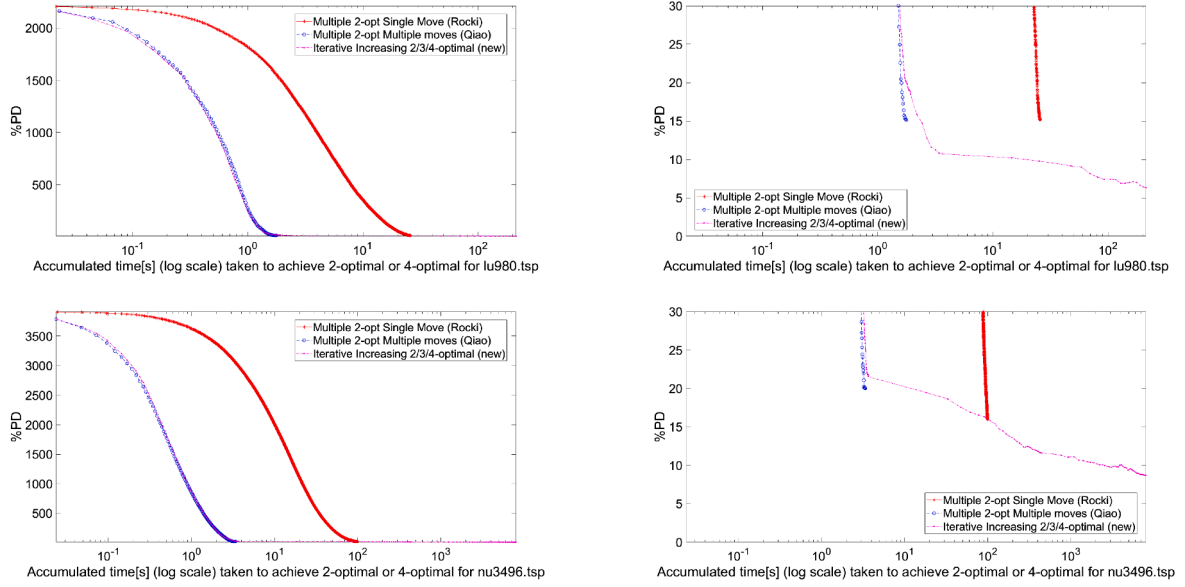
Fig. 16. Convergence dynamics for achieving λ -optimal solution using “Iterative Serial/Parallel Mixed λ -opt Search (first/best) with Multiple Variable λ -opt Moves” algorithm for the mu1979.tsp and nu3496.tsp: (left) solution quality (PD) versus computation time; (right) λ -opt search duration per iteration.

on the input quantity of n and is independent of Euclidean nearest-neighbour information. This independence leads to two implications: first, the distribution of cities in Euclidean space does not influence the performance of pure k -opt algorithms; second, it is unfair to compare them with algorithms that incorporate Euclidean nearest-neighbour information at the k -opt search stage.

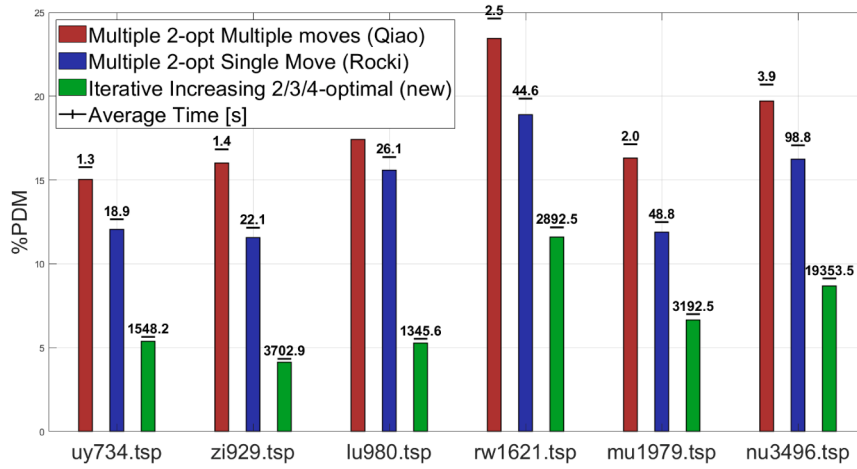
This study therefore evaluates both existing and proposed GPU-parallel algorithms performing complete k -opt search, with or without multiple k -opt moves on the same global TSP tour. Specifically examined are: (i) the original GPU-parallel 2-opt/3-opt algorithm by Rocki and Suda (2013), (ii) the ‘multiple 2-opt evaluation with multiple 2-opt

moves’ algorithm by Qiao and Créput (2020), and (iii) ‘Parallel Iterative Increasing λ -optimal (best)’ introduced in this work.”

Experimental results are recorded in Fig. 17. The proposed GPU-parallel ‘Iterative Increasing λ -optimal (best)’ algorithm achieves a higher-quality TSP solution compared to the GPU-parallel ‘multiple 2-opt evaluation multiple moves’ algorithm proposed by Qiao and Créput (2020) and the ‘multiple 2-opt evaluation single move’ algorithm proposed by Rocki and Suda (2013). Fig. 17(a) presents the convergence dynamics of the three algorithms for the TSP instance lu980.tsp and nu3496.tsp. Fig. 17(b) further validates this result across additional TSP instances in terms of TSP solution quality (PDM values).



(a) Left: complete convergence profile; Right: terminal convergence behavior.



(b) Comparison of three GPU-parallel algorithms.

Fig. 17. (a) Convergence dynamics for achieving 2-optimal or 4-optimal solutions using three algorithms for lu980.tsp and nu3496.tsp. (b) Comparison of the average computation time taken per test and TSP quality (PDM) among three algorithms.

5.3. Multi-GPU scalability assessment

As demonstrated in the above experiments, each iteration of the GPU-parallel fixed k -opt or variable λ -opt search (first/best) algorithms exhibits near-constant computation time. Additionally, because there is no inter-communication between these parallel k edge combinations' k -opt searches, the scalability of these algorithms running on multiple GPUs is determined by these algorithms' single-iteration performance.

The computation time of one iteration of parallel 3-opt and 4-opt search on 1 GPU and 2 GPUs was examined during the execution of

'iterative increasing λ -optimal (best)' algorithm for 6 large-scale TSP instances, as the difference between 1 and 2 GPUs is negligible for small-scale TSP instances. Experimental results are presented in Figs. 18 and 19.

Fig. 18 demonstrates that the computation time per iteration of the GPU-parallel complete k -opt search can be further reduced by increasing the number of GPUs. Fig. 19 illustrates the convergence dynamics of the 'iterative increasing λ -optimal (best)' algorithm using 1 and 2 GPUs for eg7146.tsp. In this test, a single iteration of serial 4-opt requires more than 48 h, while one iteration of GPU-parallel 4-opt takes an average of 790.545 s on 1 GPU and 146.732 s on 2 GPUs.

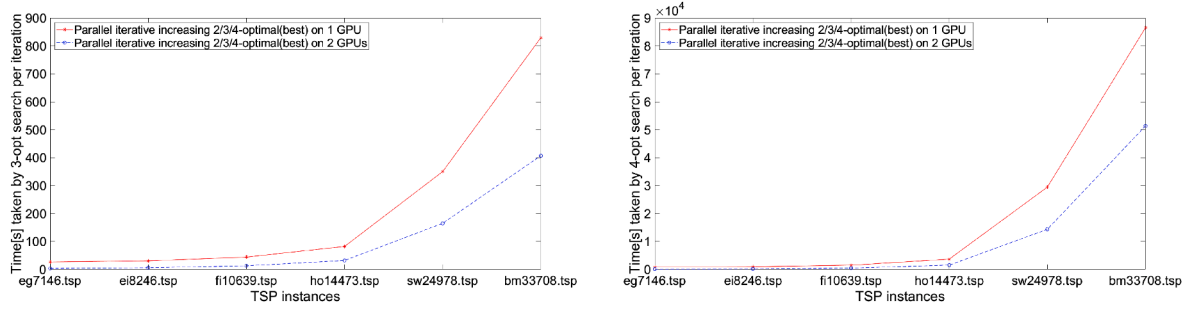


Fig. 18. The average execution time per iteration taken for GPU-parallel complete 3-opt search (left) and 4-opt search (right) implemented on either 1 or 2 GPUs during the procedure of ‘Parallel Iterative Increasing λ -optimal (best)’ across 6 large-scale TSP benchmark instances.

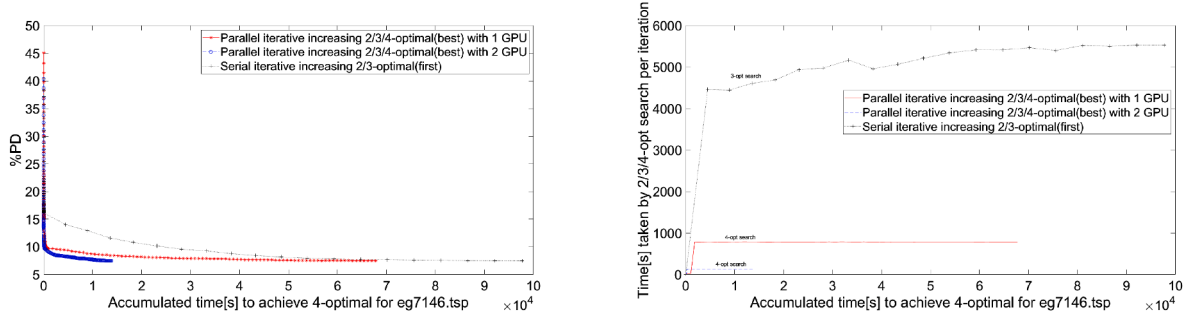


Fig. 19. Convergence dynamics for achieving 4-optimal solution using ‘parallel iterative increasing λ -optimal (best)’ on 1 and 2 GPUs for eg7146.tsp: (left) solution quality (PD) versus computation time; (right) λ -opt search duration per iteration.

6. Conclusion

This study extends GPU-parallel complete k -opt algorithms from conventional 2-opt, 3-opt to higher-order 4-opt, 5-opt, and 6-opt variants, whilst providing solutions to GPU-parallel variable λ -opt optimisation, simultaneous execution of multiple variable λ -opt moves on the same global TSP tour, and scalability across an arbitrary number of GPUs. Experimental results demonstrate that the proposed GPU-parallel algorithms achieve significant computation acceleration compared to the serial counterparts, and enable additional tour refinement beyond existing GPU-parallel 2-opt, 3-opt algorithms.

These proposed algorithms are applicable in two scenarios: (1) refining near-optimal tours through performing complete variable λ -opt ($\lambda = 2, 3, 4, 5, 6$) search on the global tour, and (2) iteratively optimising an initial TSP tour until obtaining a 4-/5-/6-optimal solution. Building upon symmetric TSP’s foundational role in combinatorial optimisation, these algorithms demonstrate universal applicability across all symmetric TSP deployment scenarios, including warehouse logistics with bidirectional travel paths, circuit board drilling sequence planning, symmetric delivery route optimisation, balanced production line scheduling.

By omitting the consideration of nearest neighbour information during k -opt opportunity searches, the proposed GPU-parallel algorithms facilitate comprehensive global optimisation across the entire TSP tour. These algorithms exhibit inherent scalability to an arbitrary number of GPUs, enabling engineers to derive globally higher-order k -optimal TSP solutions within practical timeframes by merely increasing the number of GPUs. Future work will involve investigating the GPU-parallel implementation of the LKH algorithm for large-scale TSP instances.

CRediT authorship contribution statement

Wen-Bao Qiao: Conceptualization, Methodology, Coding, Writing – original draft & review & editing; **Jean-Charles Créput:** Methodology, Writing – review & editing; **Nan Wang:** Writing – review & editing; **Kun Meng:** Writing – review & editing; **Abdelkhalek Mansouri:** Writing – review & editing.

Data availability

Code has been opensourced and its link is written in the paper.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Wen-Bao Qiao reports financial support was provided by Beijing Municipal Education Commission. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

Wen-Bao Qiao, alternatively written as Wenbao Qiao. We sincerely appreciate the valuable opinions provided by the two anonymous reviewers. This paper is sponsored by Beijing Municipal Education Commission with project ID: KM202111232002.

References

- Applegate, D. L., Bixby, R. E., Chvatal, V., & Cook, W. J. (2011). *The traveling salesman problem: A computational study*. Princeton University Press.
- Garey, M. R., & Johnson, D. S. (1979). *A guide to the theory of NP-completeness*. WH Freeman, New York, 70.
- Helsgaun, K. (2000). An effective implementation of the lin-kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1), 106–130.
- Johnson, D. S., & McGeoch, L. A. (1997). The traveling salesman problem: A case study in local optimization. *Local Search in Combinatorial Optimization*, 1, 215–310.
- Karp, R. M. (1977). Probabilistic analysis of partitioning algorithms for the traveling-salesman problem in the plane. *Mathematics of Operations Research*, 2(3), 209–224.
- Lancia, G., & Dalpasso, M. (2020). Finding the best 3-OPT move in subcubic time. *Algorithms*, 13(11), 306.
- Lancia, G., & Dalpasso, M. (2023). Orbits, schemes and dynamic programming procedures for the tsp 4-opt neighborhood. *arXiv preprint arXiv:2303.14424*.
- Laporte, G. (1992). The traveling salesman problem: An overview of exact and approximate algorithms. *European Journal of Operational Research*, 59(2), 231–247.

- Lourenço, H. R., Martin, O. C., & Stützle, T. (2003). Iterated local search. In *Handbook of metaheuristics* (pp. 320–353). Springer.
- Qiao, W.-b., & Créput, J.-c. (2017a). Massive parallel self-organizing map and 2-opt on GPU to large scale TSP. In *International work-conference on artificial neural networks* (pp. 471–482). Springer.
- Qiao, W.-B., & Créput, J.-C. (2017b). Parallel 2-opt local search on GPU. *World Academy of Science, Engineering and Technology, International Journal of Electrical, Computer, Energetic, Electronic and Communication Engineering*, 11(3), 281–285.
- Qiao, W.-B., & Créput, J.-C. (2018). Massive 2-opt and 3-opt moves with high performance gpu local search to large-scale traveling salesman problem. In *International conference on learning and intelligent optimization* (pp. 82–97). Springer.
- Qiao, W.-B., & Créput, J.-C. (2020). Multiple k- opt evaluation multiple k- opt moves with GPU high performance local search to large-scale traveling salesman problems. *Annals of Mathematics and Artificial Intelligence*, 88(4), 347–365.
- Qiao, W.-B., & Créput, J.-C. (2021). Component-based 2-/3-dimensional nearest neighbor search based on elias method to GPU parallel 2d/3d euclidean minimum spanning tree problem. *Applied Soft Computing*, 100, 106928. <https://doi.org/https://doi.org/10.1016/j.asoc.2020.106928>
- Reinelt, G. (1991). Tsplib-a traveling salesman problem library. *ORSA Journal on Computing*, 3(4), 376–384.
- Rios, E., Ochi, L. S., Boeres, C., Coelho, V. N., Coelho, I. M., & Farias, R. (2018). Exploring parallel multi-GPU local search strategies in a metaheuristic framework. *Journal of Parallel and Distributed Computing*, 111, 39–55.
- Rocki, K., & Suda, R. (2012). Accelerating 2-opt and 3-opt local search using GPU in the travelling salesman problem. In *High performance computing and simulation (HPCS), 2012 international conference on* (pp. 489–495). IEEE.
- Rocki, K., & Suda, R. (2013). High performance GPU accelerated local optimization in TSP. In *Parallel and distributed processing symposium workshops & phd forum (IPDPSW), 2013 IEEE 27th international* (pp. 1788–1796). IEEE.
- Verhoeven, M., Aarts, E. H. L., & Swinkels, P. (1995). A parallel 2-opt algorithm for the traveling salesman problem. *Future Generation Computer Systems*, 11(2), 175–182.